



Elective - III

UE23AM342BA3 : Explainable AI

Dr.Siba Ram Baral, Prof.Ayisha Noori V K

Department of Computer Science and
Engineering(AIML)

EXPLAINABLE AI

“CAN WE TRUST THE DECISIONS THAT AI MAKES?”

- This is the foundation upon which Explainable AI works.



What is Explainable AI

- With AI systems operating in safety-critical industries, AI prediction has to be justifiable and fair. In the healthcare industry when AI-powered tools detect diseases or recommend treatments, there must be complete trust and understanding of why a model suggested a certain course of action.
- It should justify the treatment plans and validate its decisions with medical evidence. Such criticality can be extrapolated to almost any industry.
- **Explainable Artificial Intelligence (AI) is the ability for artificial intelligence systems to provide understandable explanations for their decisions, recommendations, or predictions.**

Tools/Languages required

- Python Tools and Libraries
- Machine Learning Libraries (Scikit-Learn, TensorFlow/PyTorch)
- XAI Libraries: SHAP, LIME, InterpretML, OmniXAI.

Why Explainable AI Matters in 2025?



- Organizations must be fully aware of the AI decision-making process, and those that establish digital trust among consumers through explainable AI are more likely to see their annual revenue and EBIT grow at rates of 10 percent or more **(Source: McKinsey)**

Here are a few reasons why XAI matters:

- When the stakeholders and regulators understand how the AI arrived at a particular decision, it increases trust and confidence in its recommendation
- XAI can identify and correct the biases related to race, gender, age, and other factors. This results in fair outcomes
- It ensures regulatory compliance as many of them, like the EU's AI Act (GDPR) require explanations for automated decisions that affect people
- It gives context to the outcome it suggests, helping professionals make informed decisions
- When companies understand how a model works, it can help surface business interventions that might have been otherwise hidden

Why Explainable AI is the Future of Responsible Tech?

- **AI as a silent decision-maker:**

AI increasingly influences critical areas such as loan approvals, medical diagnoses, fraud detection, and autonomous systems, directly impacting human lives.

- **Trust is the central challenge:**

Explainable AI (XAI) builds transparency:

Regulatory pressure is increasing worldwide:

Laws and frameworks like the **EU AI Act**, **GDPR Article 22**, **NIST AI Risk Management Framework**, and **India's Digital India Act** mandate AI systems to be interpretable, fair, and auditable.

- **Non-compliance has serious consequences:**

Organizations that fail to meet explainability requirements risk heavy fines, legal action, and regulatory bans.

- **Growing demand for XAI expertise:**

High-stakes domains cannot rely on black-box AI:

- **Explainability is essential for AI adoption in healthcare:**

Clinicians must understand and justify AI-assisted diagnoses; lack of explainability remains a major barrier to scaling AI in healthcare (McKinsey).

- **XAI enables cross-functional collaboration:**

Explainability aligns data scientists, product managers, and compliance teams around a shared understanding of how AI systems behave.

- **A shared source of truth for responsible AI:**

XAI bridges silos, ensuring AI systems meet both performance objectives and ethical, legal, and regulatory requirements.

Emerging Roles in XAI Careers:

- **Explainable AI Engineer:**
- **AI Ethics Specialist:**
- **Data Scientist (XAI Focus):**
- **Model Governance Analyst:**
- **AI Policy and Compliance Consultant:**
- **XAI Research Scientist:**

How Explainable AI is Used Across Industries:

Healthcare:

The AI models that are used in the healthcare industry must provide clear explanations for their recommendations. The physicians must be convinced that the basis on which the AI made the decision is solid and there is no room for error. For example, the AI system can share the specific medical evidence and patient data that it used to suggest treatment to a patient. The doctor can validate the AI's reasoning and take it forward.

Finance:

Explainable AI can detect fraudulent activities by explaining how certain transactions are flagged as suspicious. When there is a clear criteria being used for its reasoning, it will ensure fairness and reduce biases. For example: It can provide banks with clear examples on why someone's loan application was rejected.

Recruitment:

There are many companies that use AI systems to screen a large volume of job applications. XAI tools can help reveal the biases (if any) that are in the AI-driven algorithms used in hiring.

Computer Vision:

Explainable AI techniques can be used to gather insights into factors that are relevant and influential in recognizing and classifying images.

Cybersecurity:

The AI algorithms that are used to detect threats must give the rationale behind each alert. With Explainable AI, professionals will be able to understand and trust the reasoning behind the alerts.

Challenges in Explainable AI:

Complexity of the AI:

One of the main challenges is the complexity of the AI itself. The AI that we talk about generally means machine learning. They become smarter when more and more data is fed into it. It requires complex mathematical models that can be difficult to translate into explanations that the average layman might not understand.

Explainability Requires a Performance Tradeoff:

Most machine algorithms are coded in a manner where it provides the results as fast and efficiently as possible. It doesn't use its resources to explain how it arrived at a particular recommendation. They achieve high accuracy by learning intricate patterns, whose complexity sacrifices transparency.

No Standard XAI Evaluation Methods:

The explanations that you receive from XAI depend on the techniques that are used. This makes it difficult to validate the correctness. Also, evaluations are domain-specific. The explanations that are suitable for the insurance industry might not meet the criteria for medical cases.

Addressing such challenges will need a consistent evaluation framework and require tailoring of expectations to specific user needs.

Future Trends in XAI:



Rise of Responsible AI Ops:

Responsible AI Ops blends AI ML Ops with frameworks for transparency and ethical risk monitoring across the entire AI lifecycle. It embeds explainability into deployment pipelines. It keeps the AI continuously accountable and adaptable to regulatory changes.

LLMs with Explainability Features:

LLMs are at the core of generative AI and automation, but the lack of transparency is concerning. Researchers are working on enhancing these models with built-in explainability modules. It provides the users with the rationale for which data influenced its answers and it even allows the users to interrogate the reasoning steps.

Explainability in Generative AI:

Generative AI is evolving into multimodal systems where it can handle all the data types together. Enterprises that deploy generative AI for design, drug discovery, or autonomous systems are working on developing layered explainability tools. The tools will show why a generative model created a particular output across modalities. This interpretability is crucial for business accountability.

Human-in-the-Loop AI Governance Models:

With Explainable AI on the rise, it is evolving into human-in-the-loop where the AI systems work with human experts. This shift is mainly because of regulatory mandates, especially in healthcare and finance.

Explainable AI is a significant component toward responsible technology.

Industry 4.0 and 5.0

XAI isn't just a feature; it's the fundamental enabler for moving from efficient, but impersonal, Industry 4.0 to a more responsible, creative, and human-focused Industry 5.0

Explainable AI-Evaluation Policy

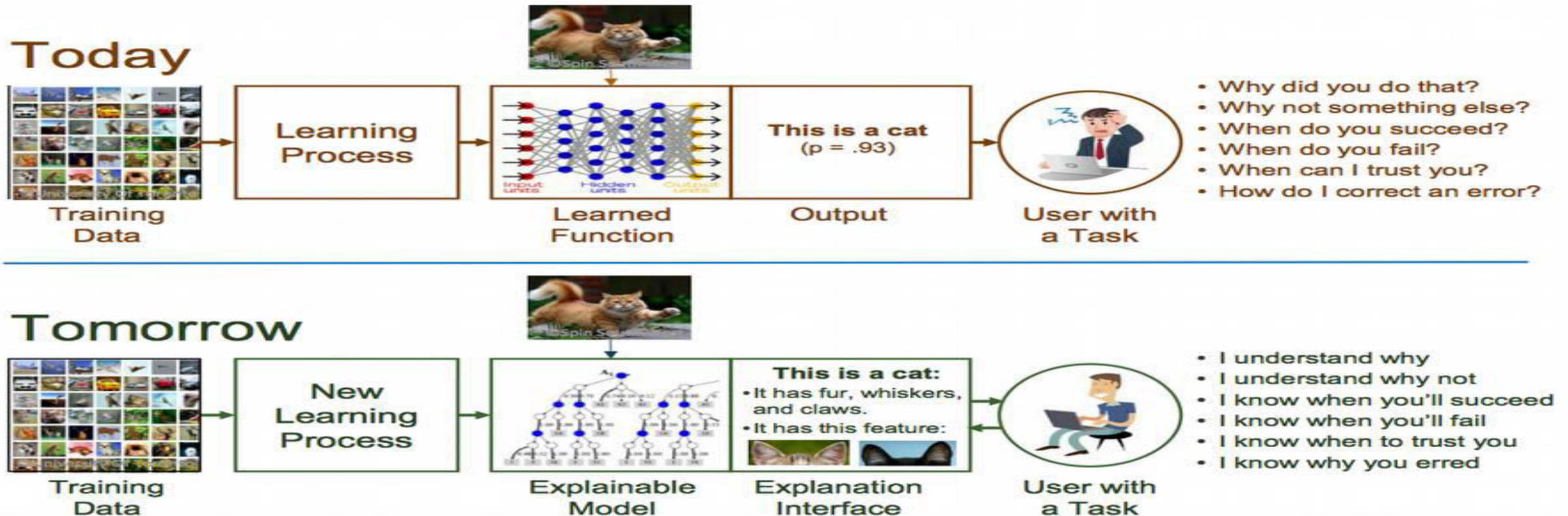
Category	Activity	Marks	Marks Reduced To
Theory	ISA 1	40	15
Theory	ISA 2	40	15
Experiential Learning Component - Banana problem	ISA - Implementation Based (conducted in the class)	4	4
Experiential Learning Component - Orange problem	ISA - Implementation Based (conducted in the class)	6	6
Experiential Learning – Jackfruit	ISA - Course Project	10	10
Theory	ESA	100	50

UNIT 1: Introduction to Interpretability and Interpretable Models

MOTIVATION

- What is Explainable AI ?

Explainable Artificial Intelligence (AI) is the ability for artificial intelligence systems to provide understandable explanations for their decisions, recommendations, or predictions



Interpretability Vs Explainability

Term	Definition	Source
Interpretability	<i>"level of understanding how the underlying (AI) technology works"</i>	ISO/IEC TR 29119-11:2020(en), 3.1.42 ^[35]
Explainability	<i>"level of understanding how the AI-based system ... came up with a given result"</i>	ISO/IEC TR 29119-11:2020(en), 3.1.31 ^[35]

Source: https://en.wikipedia.org/wiki/Explainable_artificial_intelligence

- **AI Interpretability** Focuses on understanding the innerworking of an AI Model (About Transparency of AI model)
- **AI Explainability** aims to provide reasons for models output(Justification of Model's Output)

Interpretability Vs Explainability

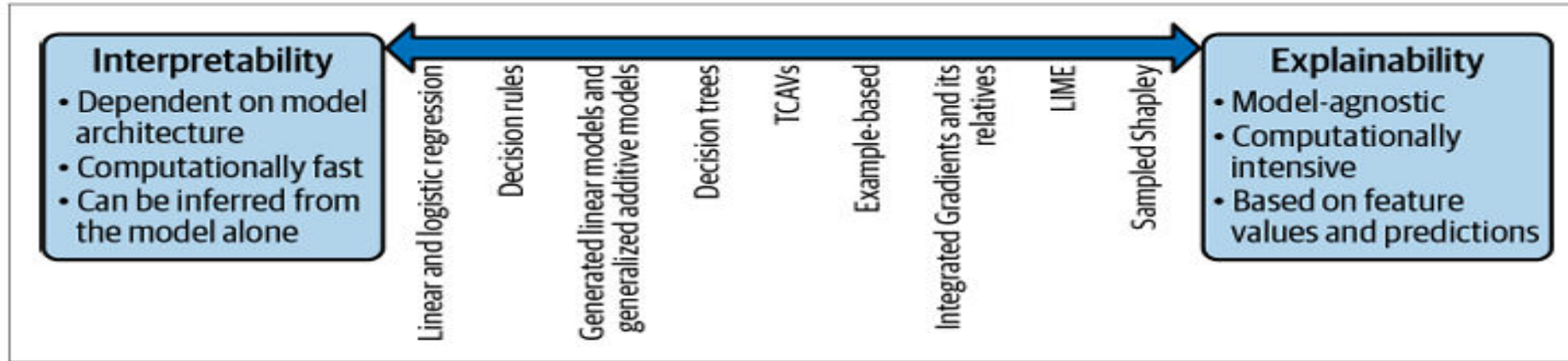


Figure 2-1. Explainability and interpretability are two ends of a spectrum. Here we show key characteristics of techniques at each end of the spectrum, and show some examples of where techniques fall along this spectrum.

- **AI Interpretability** Focuses on understanding the innerworking of an AI Model (About Transparency of AI model)
- **AI Explainability** aims to provide reasons for models output(Justification of Model's Output)

Interpretability Vs Explainability

While often used interchangeably, interpretability and explainability represent fundamentally different approaches to understanding model behavior. This distinction shapes both model selection and the tools we use to audit AI systems.

Interpretability

Direct understanding of model mechanics - the ability to comprehend how a model works by examining its structure and parameters directly. Linear regression and decision trees are inherently interpretable because humans can trace the path from input to output.

Key characteristic: Transparency is built into the model architecture itself, requiring no additional tools or post-processing.

The trade-off is clear: interpretable models offer genuine transparency but may sacrifice predictive power, while explainability tools enable complex models at the cost of indirect understanding. Your choice depends on domain requirements, regulatory constraints, and acceptable risk levels.

Explainability

Post-hoc justification of predictions - techniques applied after training to understand what a black-box model has learned. Methods like SHAP and LIME approximate model behavior without revealing internal mechanisms.

Key characteristic: External tools that provide insights into opaque models, though these explanations may be approximations.

The Interpretability Spectrum: From Transparent to Opaque

Machine learning models exist on a continuum from fully transparent to completely opaque. Understanding where your model falls on this spectrum is crucial for selecting appropriate explanation methods and setting stakeholder expectations.



Decision Trees

Completely transparent and interpretable. Every decision path is explicit and traceable. Stakeholders can follow the exact logic: "If age > 35 AND income > \$50k, then approve."



Deep Neural Networks

Black-box models that require external explanation layers. Despite high accuracy, the internal transformations across hidden layers remain opaque without tools like DeepLIFT, GradCAM, or attention visualization.

- ❑ **The DeepLIFT Framework:** Deep Learning Important FeaTures decomposes neural network predictions by comparing activations to reference values, revealing which input features most influenced the output. Unlike gradient-based methods, DeepLIFT handles activation saturation more effectively.

The interpretability gap between these model types isn't just technical—it has real-world implications for debugging, compliance, and user trust. Choose wisely based on your domain's tolerance for opacity.

Case Study: Loan Approval Decision Systems

Consider a financial institution deciding whether to approve a \$50,000 personal loan. The choice between interpretable and black-box models has profound implications for both regulatory compliance and customer trust.

Decision Tree Approach

Transparent rules for approval

- IF $\text{credit_score} \geq 720$
- AND $\text{debt_to_income} < 0.43$
- AND $\text{employment_years} \geq 2$
- THEN approve_loan

The applicant and loan officer can see exactly why the decision was made. If denied, the customer knows precisely which criteria to improve. This transparency builds trust and satisfies regulatory requirements automatically.

The decision tree sacrifices 3% accuracy for complete transparency—a worthwhile trade in regulated industries. The black-box model gains predictive power but introduces explanation complexity and potential compliance risks.

Black-Box Model Approach

Requires SHAP or LIME for explanation

A gradient boosting model or neural network might achieve 3-5% higher accuracy but offers no inherent transparency. Post-hoc explanation reveals:

- Credit score: +0.32 impact
- Recent credit inquiries: -0.18 impact
- Account age: +0.15 impact

These approximations provide insight but may not satisfy regulatory scrutiny or fully explain model behavior on edge cases.

Strategic Guidelines for Model Selection

Prioritize Interpretable Models When Possible

In high-stakes domains like healthcare, finance, and criminal justice, start with inherently interpretable models. Linear models, decision trees, and rule-based systems provide transparency that black-box models cannot match, even with sophisticated explanation tools.

Best for: Regulated industries, critical decisions, domains requiring audit trails, situations where accuracy differences are minimal

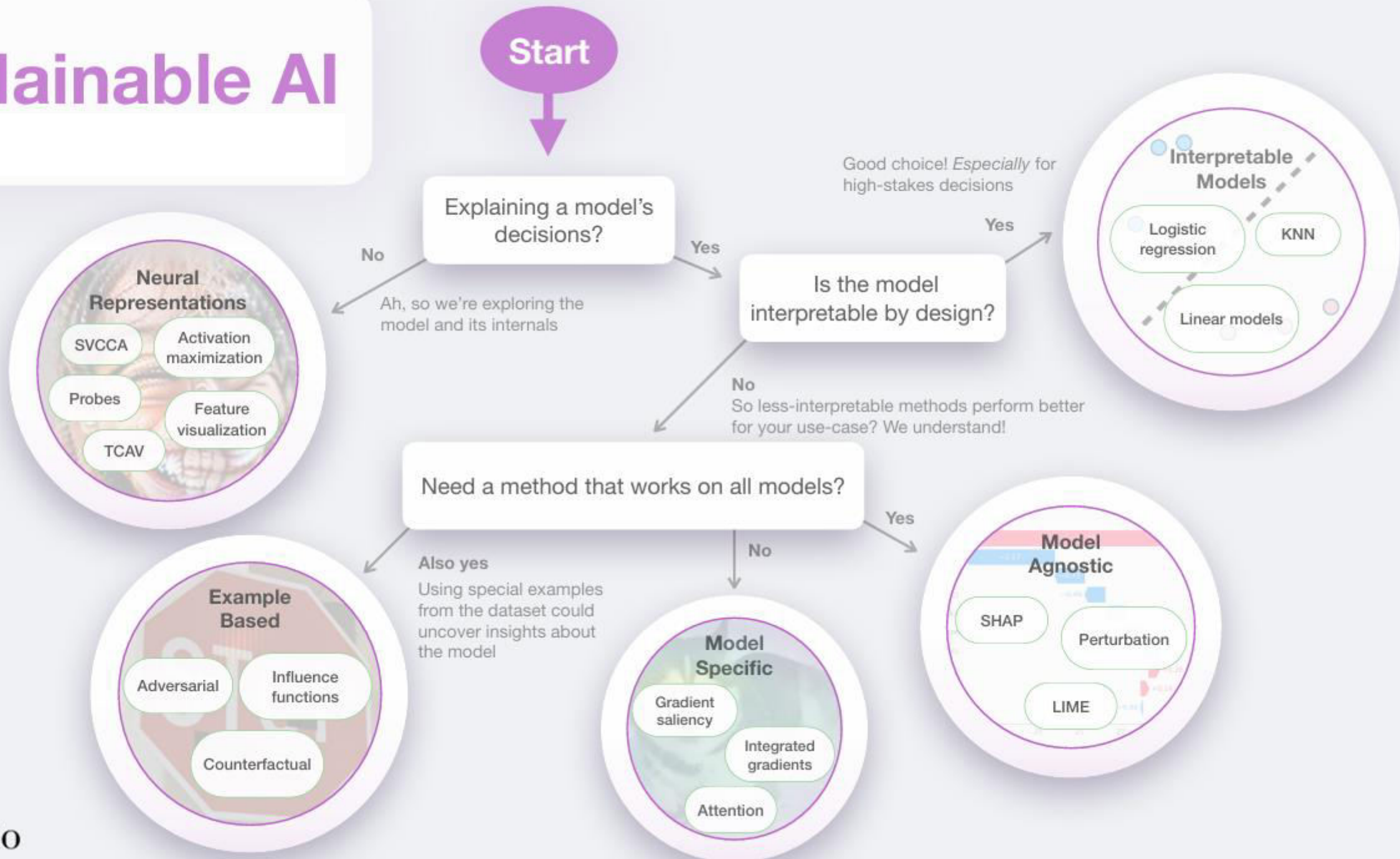
Deploy Explainability Tools for Complex Models

When interpretable models lack sufficient predictive power, leverage state-of-the-art explainability frameworks. SHAP provides theoretically grounded feature importance, LIME offers local approximations, and attention mechanisms reveal what neural networks focus on.

Best for: Computer vision, natural language processing, scenarios where accuracy gains justify complexity, research settings

"The best model is not always the most accurate one—it's the one that stakeholders can trust, regulators can audit, and practitioners can debug"

Explainable AI



Categorization in XAI

Model-agnostic

Applicable to all model types

Model-specific

Only applicable to a specific model type

Agnosticity

Global
explanation

Explaining the whole model

Local
explanation

Explaining individual predictions

Scope

Graph

Image

Text / Speech

Tabular

Data type

Visual

Feature
importance

Data points

Surrogate
models

Explanation type

Taxonomy of Interpretability Methods

Understanding the landscape of interpretability approaches requires organizing methods along multiple dimensions. This taxonomy helps practitioners select the right technique for their specific use case and constraints.



Intrinsic vs Post-hoc

Does interpretability come from the model architecture itself, or is it added after training through external analysis tools?



Global vs Local

Are we explaining the model's overall behavior across all inputs, or understanding a single specific prediction?



Transparent vs Black-box

Can humans directly understand the model's decision process, or do we need approximation methods to peek inside?

These dimensions are not mutually exclusive—a decision tree is both intrinsic and transparent, while SHAP provides post-hoc explanations that can be either global or local. Understanding these categories empowers you to match explanation methods to stakeholder needs and technical constraints.

Intrinsic vs Post-hoc Interpretability

The fundamental divide in interpretability approaches centers on *when* and *how* we achieve understanding of model behavior. This distinction affects everything from model selection to audit procedures.

Intrinsic : Interpretable by design

The model structure itself is transparent and human-comprehensible. Examples include linear regression, decision trees, rule-based systems, and sparse additive models.

- No additional tools needed
- Explanation is exact, not approximate
- Often preferred for regulated domains

1

2

Post-Hoc: Explanation generated after prediction

External methods analyze trained models to approximate their behavior. Techniques include SHAP, LIME, attention visualization, and saliency maps.

- Enables complex model architectures
- Explanations may be approximations
- Requires additional computational overhead

Trade-offs to Consider

Intrinsic models guarantee fidelity—the explanation is the model. However, they may sacrifice predictive accuracy for transparency.

Post-hoc methods unlock powerful architectures but introduce approximation error. The explanation might not fully capture model complexity or edge case behavior.

Global vs Local Explanations: Scope Matters

Beyond the intrinsic versus post-hoc distinction, interpretability methods differ in their scope of analysis. Understanding whether you need global or local explanations shapes your choice of tools and techniques.



Global Explanations

Understanding entire model behavior

Global methods reveal how the model behaves across the entire feature space and dataset. They answer questions like "What features matter most overall?" and "What patterns has the model learned?"

Examples: Feature importance rankings, partial dependence plots, global SHAP values, model-wide rule extraction

Use cases: Model debugging, bias auditing, regulatory compliance, stakeholder communication, scientific discovery



Local Explanations

Understanding individual predictions

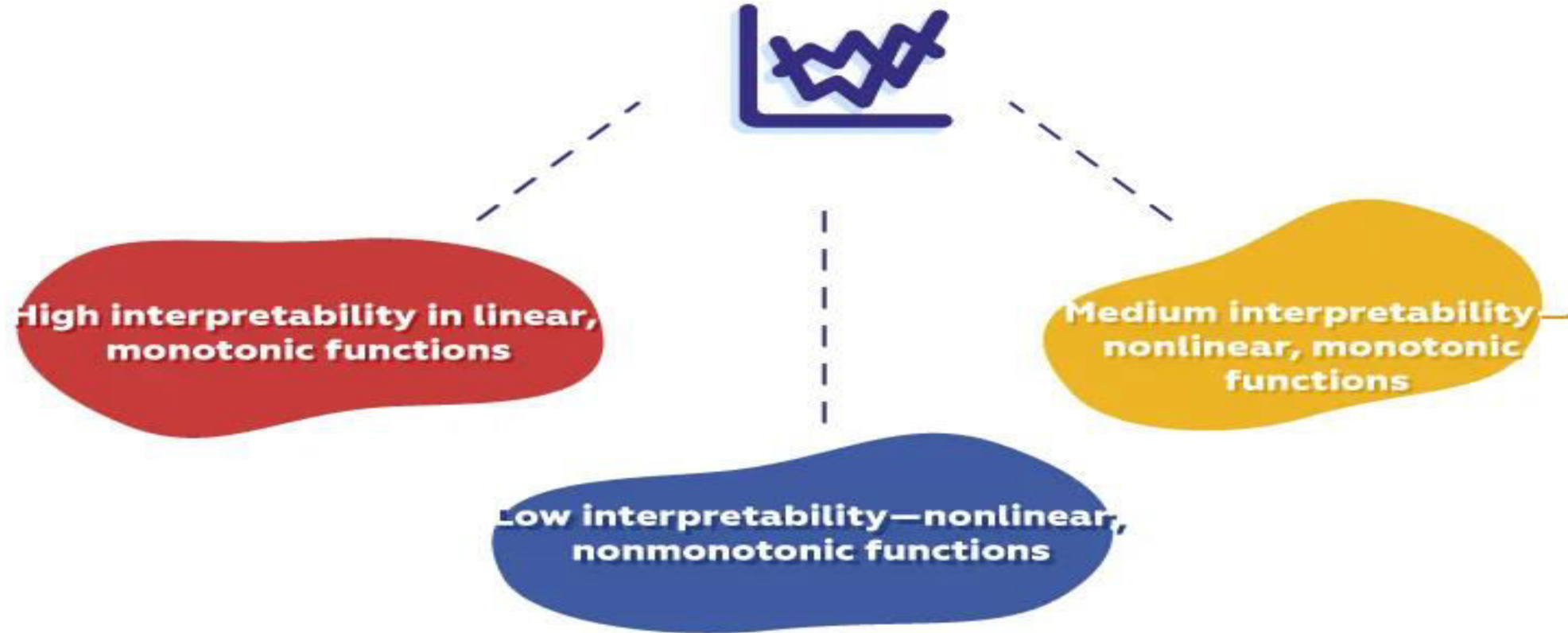
Local methods explain why the model made a specific prediction for a particular instance. They answer "Why was this loan denied?" or "What made this transaction flagged as fraudulent?"

Examples: LIME, individual SHAP values, counterfactual explanations, attention weights, influence functions

Use cases: Customer explanations, appeal processes, debugging specific failures, personalized recommendations

📌 **Complementary Approaches:** The most robust interpretability strategies combine both global and local methods. Global analysis identifies systemic issues and overall model behavior, while local explanations help users understand specific decisions that affect them directly.

Model complexity and scale of interpretability



Case Study: Decision Trees vs SHAP Explanations in Practice

Let's examine a real-world scenario comparing intrinsic interpretability against post-hoc explainability, highlighting the practical trade-offs that data scientists face when deploying models to production.

Decision Tree Model

Performance: 82% accuracy

Interpretability: Complete transparency

- 15 decision rules easily understood by stakeholders
- Zero explanation overhead
- Instant regulatory compliance
- Customers can contest decisions with clear criteria
- Model debugging takes minutes

Limitation: Cannot capture complex non-linear interactions that improve accuracy **Advantage:** Captures subtle patterns that simpler models miss

Gradient Boosting + SHAP

Performance: 89% accuracy

Interpretability: Post-hoc approximation

- 7% accuracy improvement
- SHAP values require 2-5 seconds per prediction
- Explanations may vary with background dataset
- Harder to validate explanation fidelity
- Additional complexity in deployment

7%

Accuracy Gain

Performance improvement from complex model

100%

Transparency

Decision tree offers complete interpretability

5s

Explanation Time

Additional latency for SHAP computation

The Verdict: In regulated financial services, the decision tree's transparency often outweighs the accuracy gain. For recommendation systems or fraud detection where false negatives are costly, the complex model with explanations may be justified. Context determines the right choice.

Evaluations of Interpretability

Evaluating machine learning interpretability involves assessing how well humans can understand a model's decisions, often through **application-grounded** (real-world tasks), **human-grounded** (user studies comparing explanations), or **functionally-grounded** (proxy metrics like rule count) methods

Application-grounded evaluation

(real task)

Requires a human to perform experiments within a real-life application.

For example, to evaluate an interpretation on diagnosing a certain disease, the best way is for the doctor to perform diagnoses.

Human-grounded evaluation

(simple task)

is about conducting simpler human-subject experiments.

For example, humans are presented with pairs of explanations, and must choose the one that they find to be of higher quality.

Functionally grounded evaluation

Requires no human experiments.

Instead, it uses a proxy for explanation quality.

This method is much less costly than the previous two. The challenge, of course, is to determine what proxies to use.

For example, decision trees have been considered interpretable in many situations, but additional research is required.

Scope of Interpretability

The scope of interpretability in artificial intelligence and machine learning defines the extent and level at which a human can understand a model's inner workings, its decision-making process, or the rationale behind a specific prediction. It ranges from understanding the entire system's logic to explaining a single outcome and is vital for building trust, ensuring fairness, meeting regulations, and debugging models.

- **Algorithm Transparency**

How does the algorithm create the model?

- **Global, Holistic Model Interpretability**

How does the trained model make predictions?

- **Global Model Interpretability on a Modular Level**

How do parts of the model affect predictions?

- **Local Interpretability for a Single Prediction**

Why did the model make a certain prediction for an instance?

- **Local Interpretability for a Group of Predictions** Why did the model make specific predictions for a group of instances?



THANK YOU



Elective - III

UE23AM342BA3 : Explainable AI

INTERPRETABLE MODELS

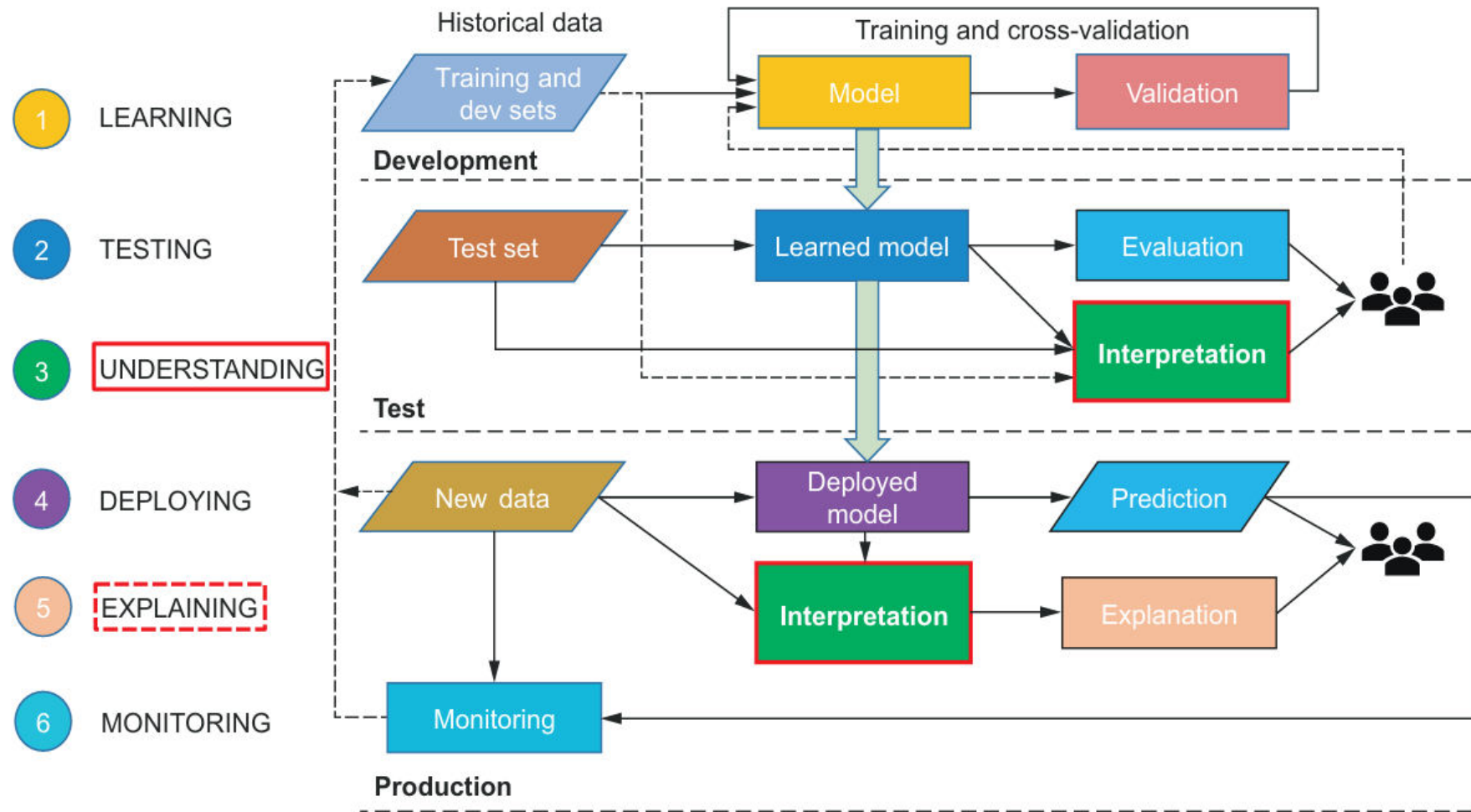
Dr.Siba Ram Baral, Prof.Ayisha Noori V K

Department of Computer Science and
Engineering(AIML)

Interpretable Models:

WHITE-BOX MODELS : INHERENTLY TRANSPARENT AND INTERPRETABLE

How to build a robust, interpretable AI system.?



Whitebox models:

White-box models are inherently transparent, and the characteristics that make them transparent are:

- The algorithm used for machine learning is straightforward to understand, and we can clearly interpret how the input features are transformed into the output or target variable.
- We can identify the most important features to predict the target variable, and those features are understandable
- **Eg: Linear regression, Logistic Regression, Decision Trees, GAM(generalized additive models), GLM(Generalized Linear Models)**

White-box model	Machine learning task(s)
Linear regression	Regression
Logistic regression	Classification
Decision trees	Regression and classification
GAMs	Regression and classification

Linear Regression:

- A linear regression model predicts the target as a weighted sum of the feature inputs. The linearity of the learned relationship makes the interpretation easy. Linear models can be used to model the dependence of a regression target y on some features x .
- The learned relationships are linear and can be written for a single instance i as follows:

$$\begin{aligned} y &= w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n \\ &= w_0 + \sum_{i=1}^n w_ix_i \end{aligned}$$

The objective of the linear regression learning algorithm is to determine the weights that accurately predict the target variable for all patients in the training set. We can apply the following techniques here:

- **Gradient descent**
- **Closed-form solution (e.g., the Newton equation)**

Linear Regression : Example

- A linear regression model can be easily trained using the Scikit-Learn package in Python. The code to train the model is shown next. Note that the open diabetes dataset provided by Scikit-Learn is used here

Imports numpy to evaluate the performance of model

Imports the scikit-learn class for linear regression

Imports the scikit-learn function to split the data into training and test sets

```
➤ from sklearn.model_selection import train_test_split  
➤ from sklearn.linear_model import LinearRegression  
➤ import numpy as np
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y,  
                                                    test_size=0.2,  
                                                    random_state=42)
```

Splits the data into training and test sets, where 80% of the data is used for training and 20% of the data for testing, and ensures that the seed for the random-number generator is set using the random_state parameter to ensure consistent train-test splits

```
lr_model = LinearRegression()
```

Initializes the linear regression model, which is based on least squares

```
lr_model.fit(X_train, y_train)
```

Learns the weights for the model by fitting on the training set

```
y_pred = lr_model.predict(X_test)
```

```
➤ mae = np.mean(np.abs(y_test - y_pred))
```

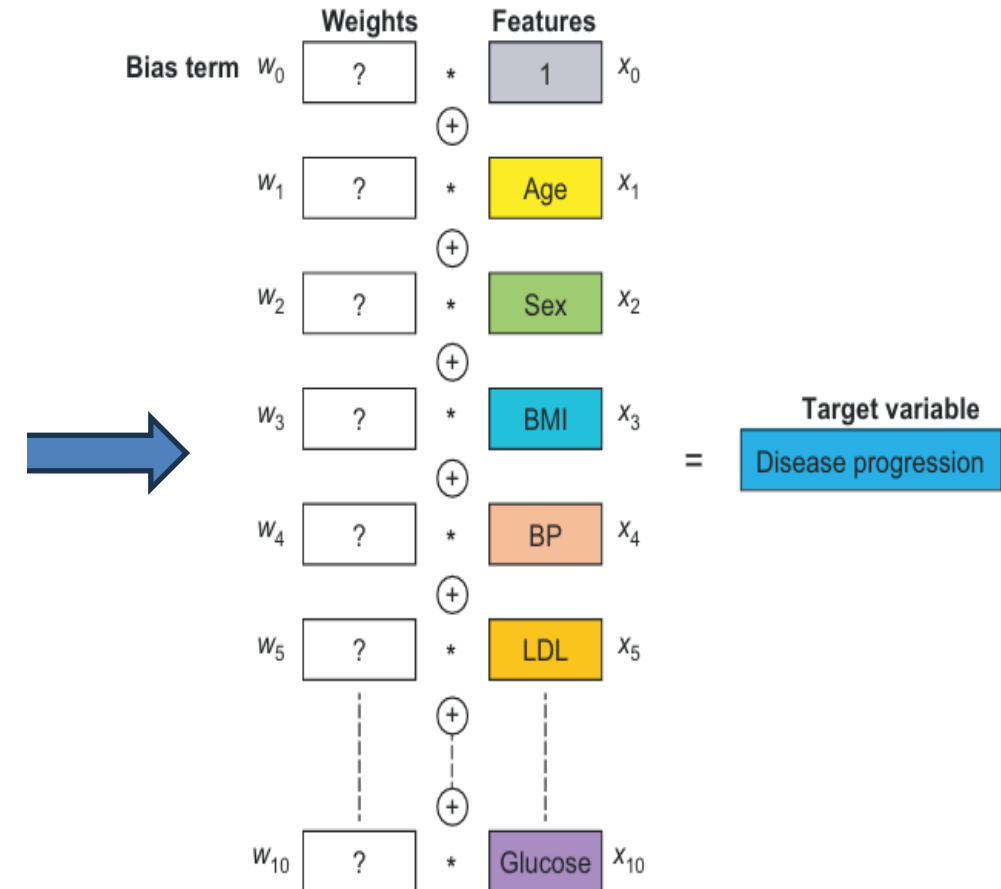
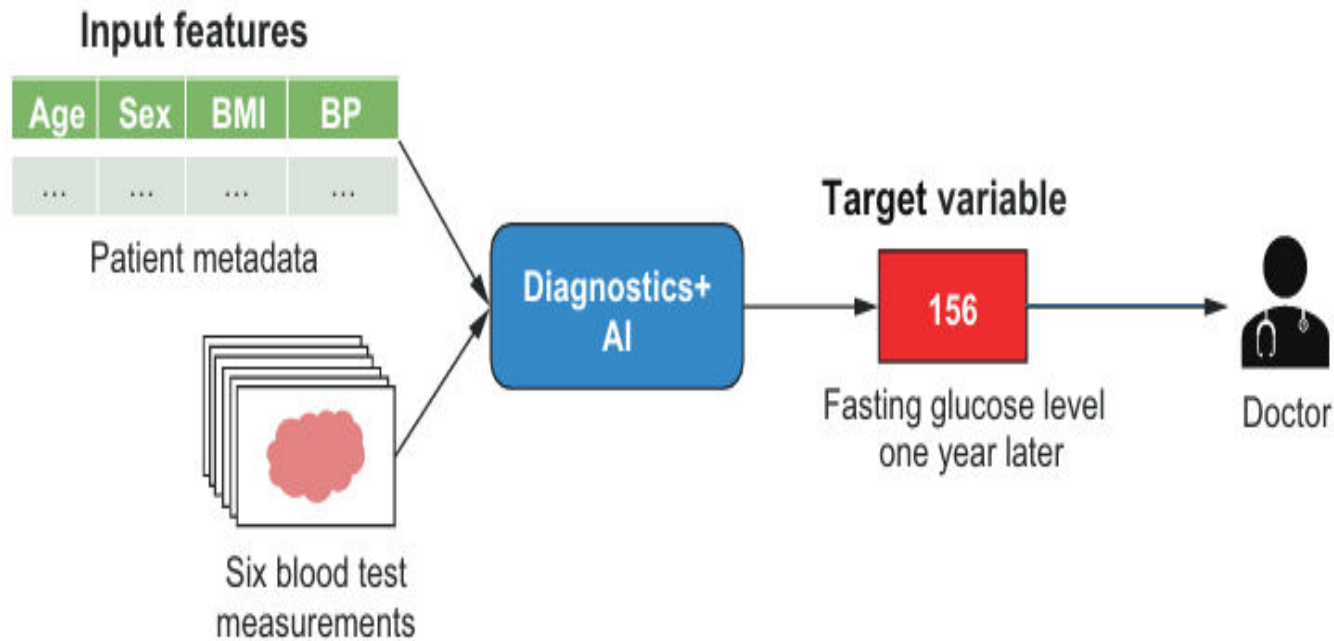
Evaluates the model performance using the mean absolute error (MAE) metric

Uses the learned weights to predict the disease progression for patients in the test set

Interpreting linear regression : Case study

Diabetes Prediction Example:

Disease Progression represented as linear combination of 10 features



Interpreting linear regression : Case study

Diabetes Prediction Example:

```
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
sns.set(style='whitegrid')
sns.set_palette('bright')

weights = lr_model.coef_

feature_importance_idx = np.argsort(np.abs(weights))[:, -1]
feature_importance = [feature_names[idx].upper() for idx in
    feature_importance_idx]
feature_importance_values = [weights[idx] for idx in
    feature_importance_idx]

f, ax = plt.subplots(figsize=(10, 8))
sns.barplot(x=feature_importance_values, y=feature_importance, ax=ax)
ax.grid(True)
ax.set_xlabel('Feature Weights')
ax.set_ylabel('Features')
```

Imports numpy to perform operation on vectors in an optimized way

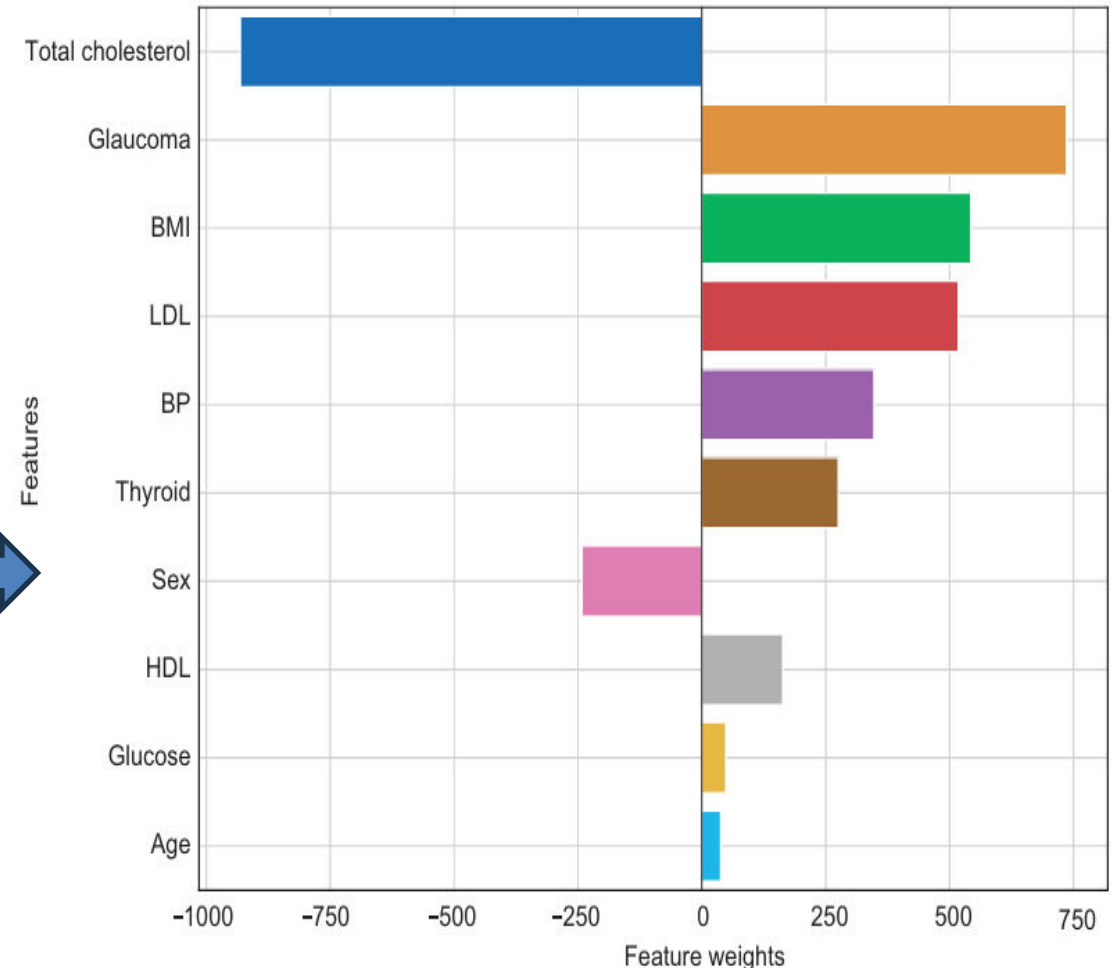
Imports matplotlib and seaborn to plot the feature importance

Obtains the weights from the linear regression model trained earlier using the coef_ parameter

Sorts the weights in descending order of importance and gets their indices

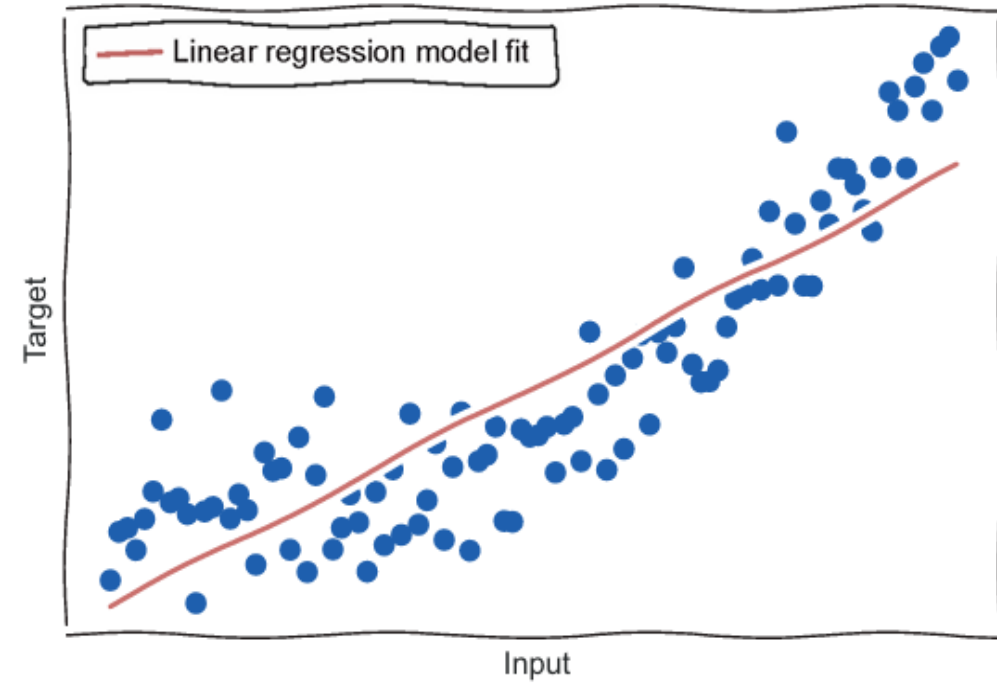
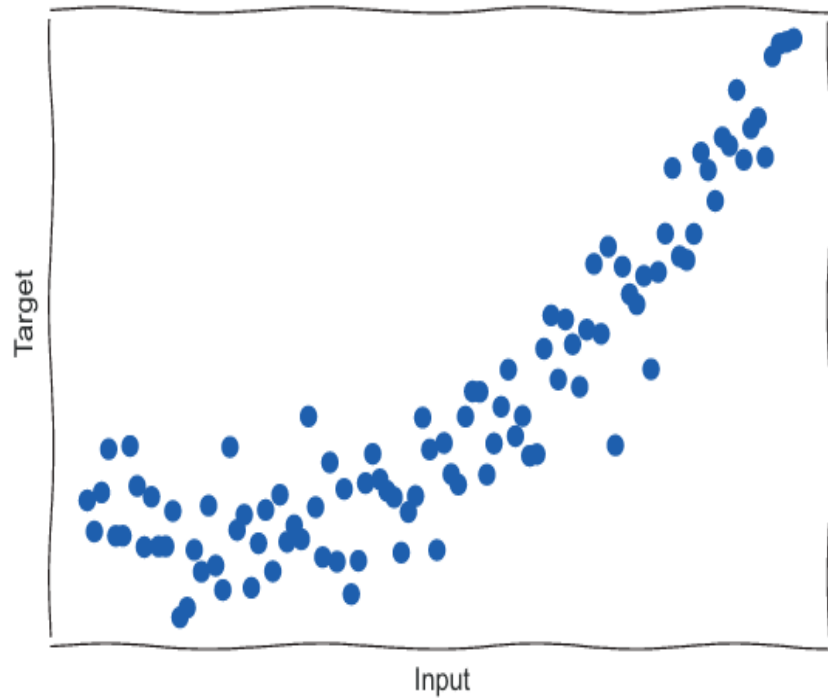
Uses the ordered indices to get the feature names and the corresponding weight values

Generates the plot shown in figure 2.7



Limitations of Linear Regression :

- It is highly transparent and easy to understand.
- However, it has poor predictive power, especially in cases where the relationship between the input features and target is **non-linear**



Linear Regression

Use Case 2: House Price Prediction

Predicting residential property values using interpretable features that real estate professionals and buyers can understand.

Input Features:

- Square footage
- Neighborhood location score
- Number of bedrooms and bathrooms
- Property age

Output: Predicted price in USD

```
from sklearn.linear_model import LinearRegression
```

```
model = LinearRegression()  
model.fit(X_train, y_train)
```

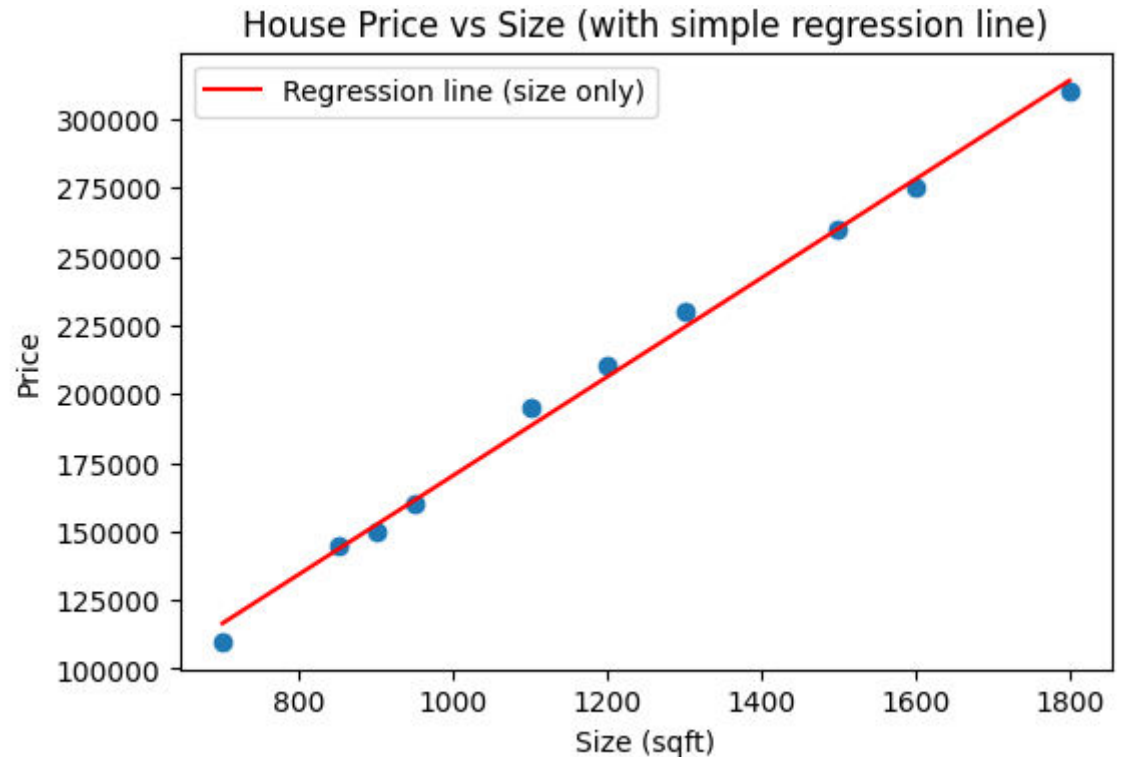
```
# Interpret coefficients
```

```
print(model.coef_)
```

```
# [245.3, 18500, 15000, -2300]
```

```
# Each sqft adds $245
```

```
# Each room adds $15k
```



The coefficient for square footage (245.3) tells us that each additional square foot increases the predicted price by approximately \$245, holding all other features constant.

Logistic Regression: Binary Outcomes

Logistic regression extends linear models to classification problems by predicting the probability of binary outcomes. It uses the sigmoid function to transform linear combinations into probabilities between 0 and 1.

Linear Combination

Compute weighted sum of features: $z = \beta_0 + \beta_1 x_1 + \dots$

Sigmoid Transform

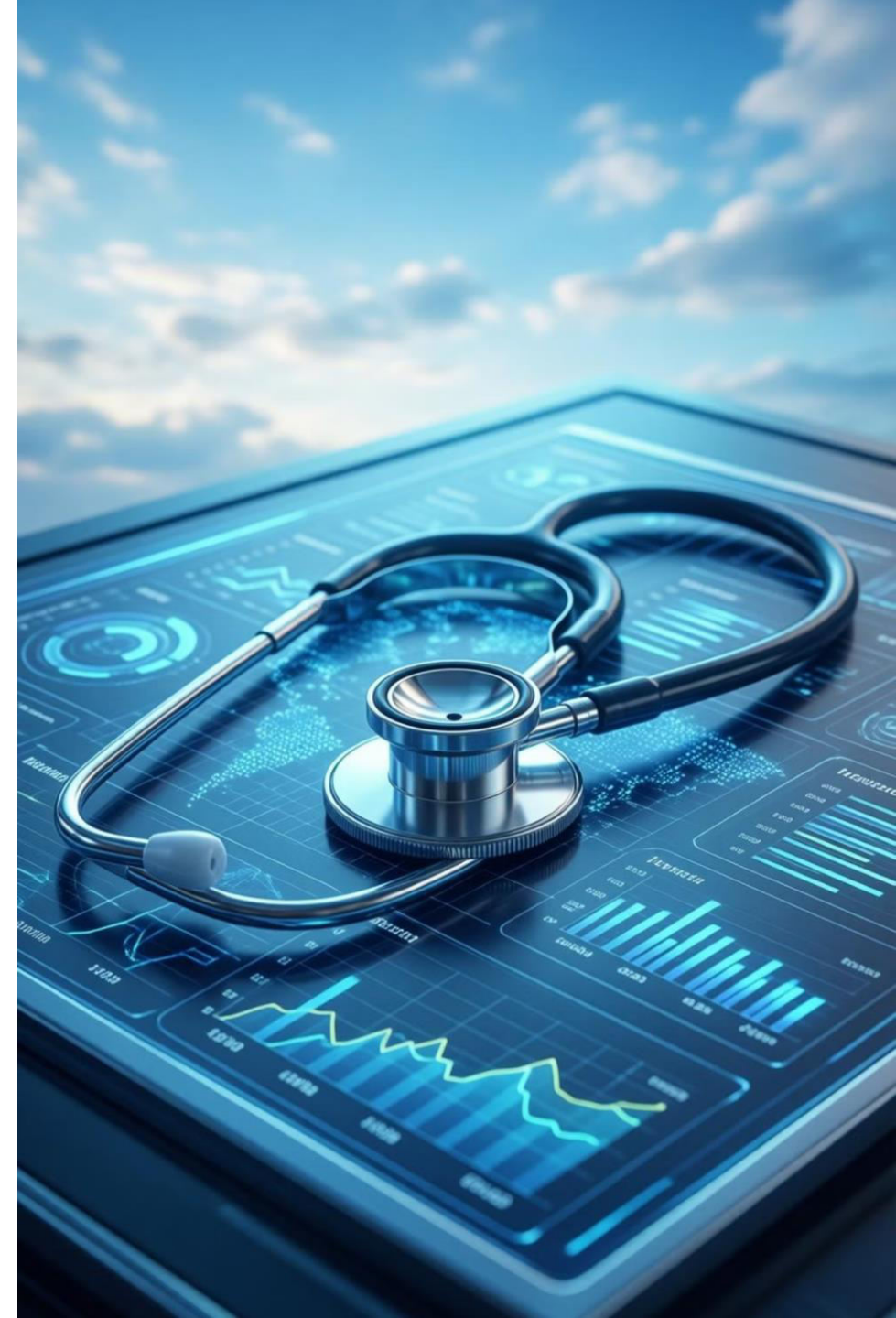
Map to probability space using sigmoid function

Interpret Log-Odds

Coefficients represent change in log-odds for unit increase in feature

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-z}}$$

The sigmoid function ensures outputs are valid probabilities



Logistic Regression Example

Disease Risk Prediction:

Estimating the probability of disease onset based on patient health indicators. This application is common in clinical decision support systems.

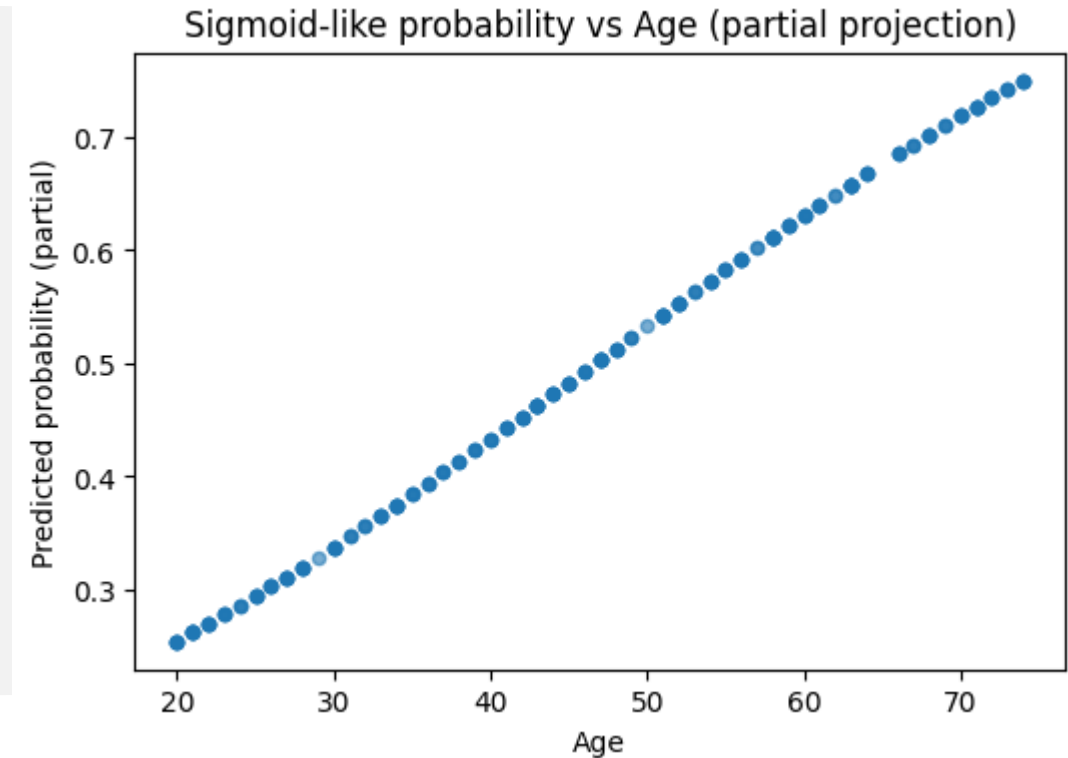
Patient Features:

- Age (years)
- Body Mass Index (BMI)
- Systolic blood pressure
- Family history (binary)

Output: Probability of disease (0 to 1)

```
from sklearn.linear_model import  
LogisticRegression  
  
model = LogisticRegression()  
model.fit(X_train, y_train)  
  
# Coefficients as log-odds  
print(model.coef_)  
# [[0.05, 0.12, 0.03, 0.89]]  
  
# Predict probability  
prob = model.predict_proba(X_test)  
print(prob[:, 1]) # P(disease)
```

A coefficient of 0.05 for age means each additional year increases the log-odds of disease by 0.05, corresponding to a $e^{0.05} \approx 1.05$ multiplicative increase in odds.



Generalized Linear Models (GLM)

GLMs extend the linear modeling framework by introducing link functions that connect the linear predictor to the response variable. This flexibility enables modeling diverse data types while maintaining interpretability.



Common Link Functions

- **Logit:** Binary classification
- **Log:** Count data (Poisson regression)
- **Identity:** Continuous outcomes (standard linear)
- **Probit:** Alternative for binary outcomes

Ideal Applications

- Insurance claim frequency modeling
- Epidemiological count data
- Credit risk assessment
- Quality control defect rates

Key Components Contributing to GLM Interpretability:

- **Linear Predictor:** The core of a GLM is a linear combination of input features and their corresponding coefficients (β weights)).
- **Coefficients (Weights):** The magnitude and sign of the coefficients directly indicate the direction and strength of each feature's impact on the predicted outcome. For example, in a logistic regression (a type of GLM), a positive coefficient means that increasing the feature's value increases the probability of the positive class.
- **Link Function:** This function connects the linear predictor to the expected value of the response variable (e.g., using a logit link for probabilities between 0 and 1, or a log link for positive count data). By inverting the link function, the effect of features on the expected outcome can be directly interpreted (e.g., a one-unit increase in a feature multiplies the expected count by a specific factor).
- **Assumed Distribution:** GLMs require specifying a probability distribution (from the exponential family, such as Gaussian, Poisson, or Binomial) for the target variable. This assumption is transparent and guides how the model should be interpreted in context.

GLM Example: Insurance Risk



Predicting Claim Probability

Insurance companies use GLMs with binomial family and logit link to model the probability of claims, enabling accurate premium pricing and risk stratification.

```
import statsmodels.api as sm

# Binomial family with logit link
glm = sm.GLM(y, X,
             family=sm.families.Binomial())

result = glm.fit()
print(result.summary())

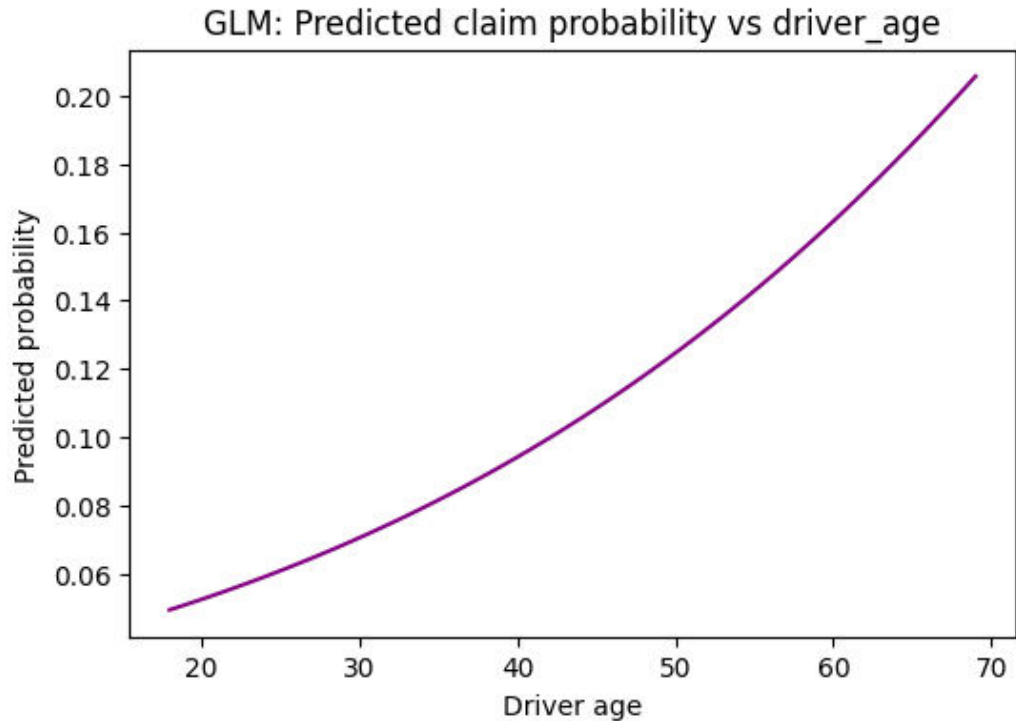
# Coefficient interpretation
# exp(coef) = odds ratio
```

Model Features:

- Driver age and experience
- Vehicle type and age
- Claims history
- Geographic region

Why GLM?

The logit link naturally handles probability bounds (0-1) while binomial distribution matches the binary claim/no-claim outcome structure.



Generalized Linear Model Regression Results

Dep. Variable:	claim	No. Observations:	400			
Model:	GLM	Df Residuals:	396			
Model Family:	Binomial	Df Model:	3			
Link Function:	Logit	Scale:	1.0000			
Method:	IRLS	Log-Likelihood:	-138.34			
Date:	Mon, 12 Jan 2026	Deviance:	276.68			
Time:	17:30:21	Pearson chi2:	414.			
No. Iterations:	5	Pseudo R-squ. (CS):	0.05071			
Covariance Type:	nonrobust					
=====						
	coef	std err	z	P> z	[0.025	0.975]

const	-5.5266	0.915	-6.039	0.000	-7.320	-3.733
driver_age	0.0315	0.011	2.909	0.004	0.010	0.053
annual_mileage	0.0001	5.32e-05	2.752	0.006	4.21e-05	0.000
prior_accidents	0.4349	0.209	2.084	0.037	0.026	0.844
=====						

Generalized Additive Models (GAM)

GAMs maintain an additive structure like linear models but allow each feature to have its own flexible, smooth function. This captures non-linear relationships while preserving interpretability through visualization of individual feature effects.

$$y = \beta_0 + f_1(x_1) + f_2(x_2) + f_3(x_3) + \dots + f_n(x_n)$$

Each f_i is a smooth, non-parametric function learned from data



Non-Linear Flexibility

Captures complex patterns like U-shapes, plateaus, and thresholds that linear models miss.



Visual Interpretability

Plot each feature's partial effect to see exactly how it influences predictions across its range.



Additive Structure

Effects combine additively, making it easy to understand individual feature contributions.

GAM Example: Customer Churn

Predicting customer churn with GAMs reveals non-linear relationships that business stakeholders can visualize and act upon. For instance, churn risk might decrease rapidly at first with tenure, then plateau after 12 months.



Key Features:

- Customer tenure (months)
- Monthly charges (\$)
- Contract type (ordinal)
- Support tickets (count)

Interpretable Insights:

The smooth curves reveal actionable patterns—churn spikes when monthly monthly charges exceed \$80, but long-term contracts show protective effects regardless of price.

```
GAM summary:
LogisticGAM
=====
Distribution:      BinomialDist Effective DoF:      19.8747
Link Function:    LogitLink Log Likelihood:        -307.165
Number of Samples: 500 AIC:      654.0795
                  AICc:      655.9896
                  UBRE:      3.34
                  Scale:      1.0
                  Pseudo R-Squared: 0.0749
=====
Feature Function   Lambda      Rank      EDof      P > x      Sig. Code
=====
s(0)              [0.6]      20       10.4     4.05e-04   ***
s(1)              [0.6]      20        7.6     7.91e-01
f(2)              [0.6]      3         1.9     2.02e-01
intercept         1         0.0      7.32e-01
=====
Significance codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

WARNING: Fitting splines and a linear function to a feature introduces a model identifiability problem
which can cause p-values to appear significant when they are not.

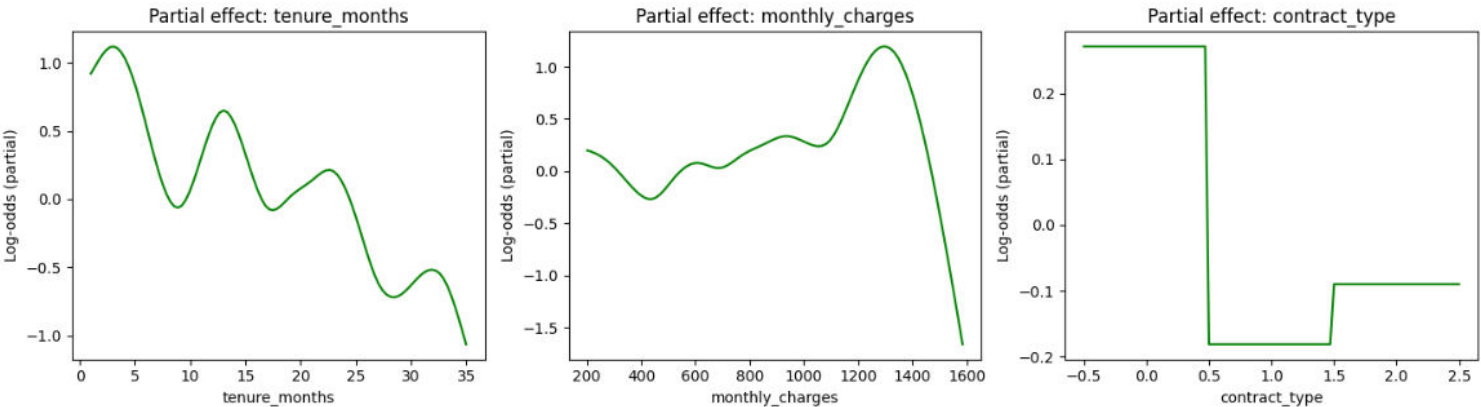
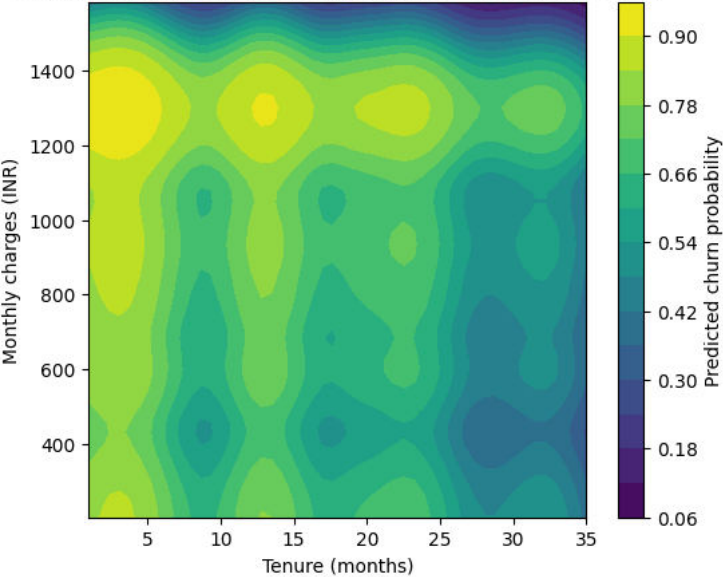
WARNING: p-values calculated in this manner behave correctly for un-penalized models or models with
known smoothing parameters, but when smoothing parameters have been estimated, the p-values
are typically lower than they should be, meaning that the tests reject the null too readily.

None
/tmp/ipython-input-325674001.py:16: UserWarning: KNOWN BUG: p-values computed in this summary are likely much smaller than they should be.

Please do not make inferences based on these values!

Collaborate on a solution, and stay up to date at:
github.com/dswah/pyGAM/issues/163
```

GAM probability surface (contract_type=month-to-month)





THANK YOU



Elective - III

UE23AM342BA3 : Explainable AI

Banana Problem – Unit 1

Prof.Ayisha Noori V K

Department of Computer Science and
Engineering(AI&ML)

Contents:

- Banana Problem Statement
- Submission Guidelines
- Submission Link

Banana Problem –Unit1 Individual Activity (10 Marks)

Problem Title:

Explaining Disease Risk Prediction using Generalized Additive Models (GAM)

Problem Statement:

Design a **Python program** that loads the `disease_risk.csv` dataset, trains a **Generalized Additive Model (GAM)** to predict disease risk, and explains **why the model predicted “disease” for a selected patient**. Visualize the **feature-wise contributions** influencing the prediction.

Tasks to be Performed

1. Load and preprocess the `disease_risk.csv` dataset.
2. Train a **GAM model** for disease prediction.
3. Select one test instance for which the model predicts “**disease.**”
4. Explain the prediction using the **intrinsic interpretability** of the GAM by:
 - Identifying important contributing features.
 - Showing whether each feature increases or decreases disease risk.
5. Visualize:
 - Feature contribution plot for the selected patient.
 - (Optional) Global shape plots to understand overall feature behavior.
6. Provide a **brief interpretation** of the explanation in simple language.

Expected Output



1. Trained GAM model
2. Feature contribution visualization for one patient
3. Short written explanation of the prediction

Marks Distribution (10 Marks)

Component	Marks
Data loading and preprocessing	2
Correct GAM implementation	3
Explanation of individual prediction	3
Visualization and interpretation	2

Install required library : **pip install pygam**

Submission Link : <https://forms.gle/hNw5CrQ4J2MC3VTz7>



THANK YOU



Elective - III

UE23AM342BA3 : Explainable AI

Decision Tree and Rule Based Methods

Prof.Ayisha Noori V K

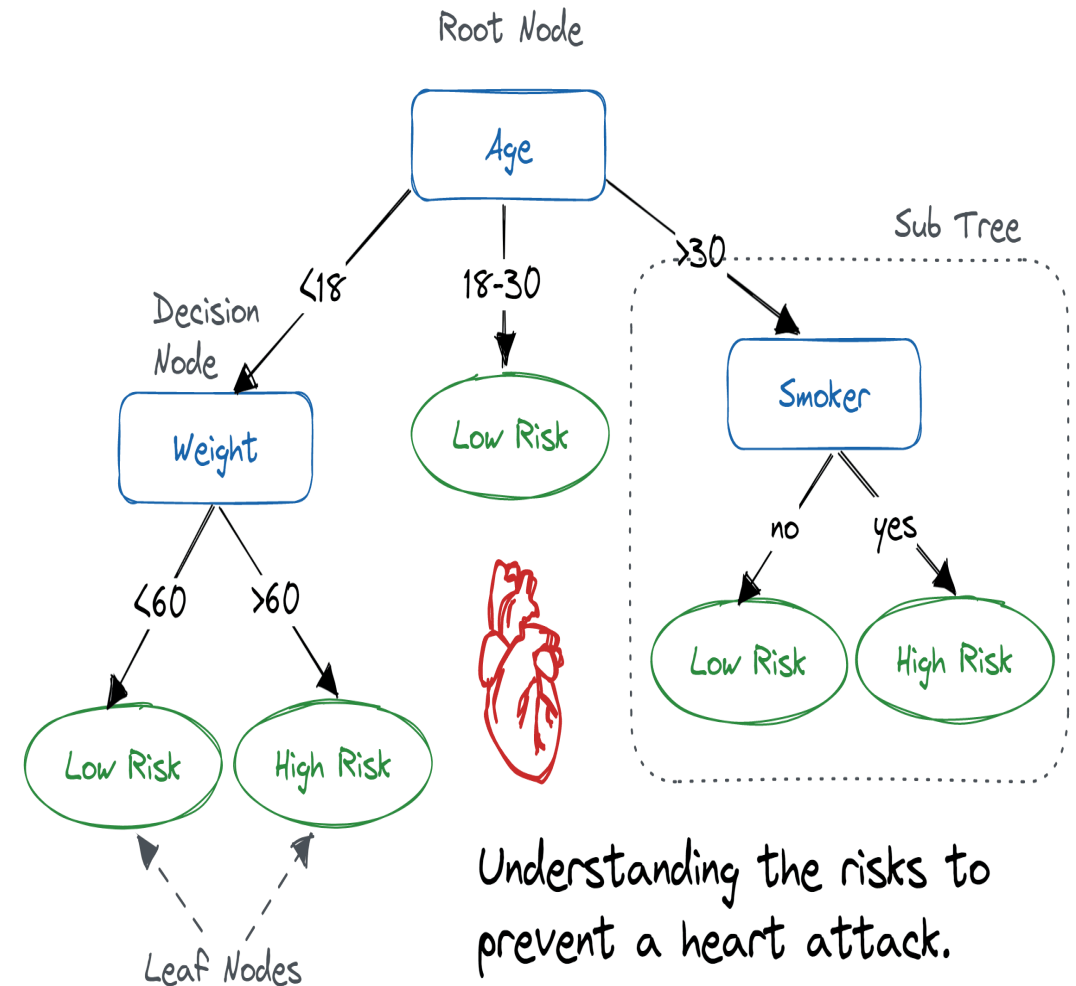
**Department of Computer Science and
Engineering(AI&ML)**

Contents:

- Decision Tree
- Rule Based Methods
- Anchors
- Counterfactual explanations
- Structured Data Explainers
- Recap of Unit1

DECISION TREE:

- A decision tree is a great machine learning algorithm that can be used to model complex nonlinear relationships. It can be applied to both regression and classification tasks.
- It has relatively higher predictive power than linear regression and is highly interpretable, too
- The basic idea behind a decision tree is to find optimum splits in the data that best predict the output or target variable.



Decision Tree (Disease Risk Dataset)



Concept: Decision Tree Classifier

Dataset: disease_risk.csv (age, BMI, systolic BP, disease)

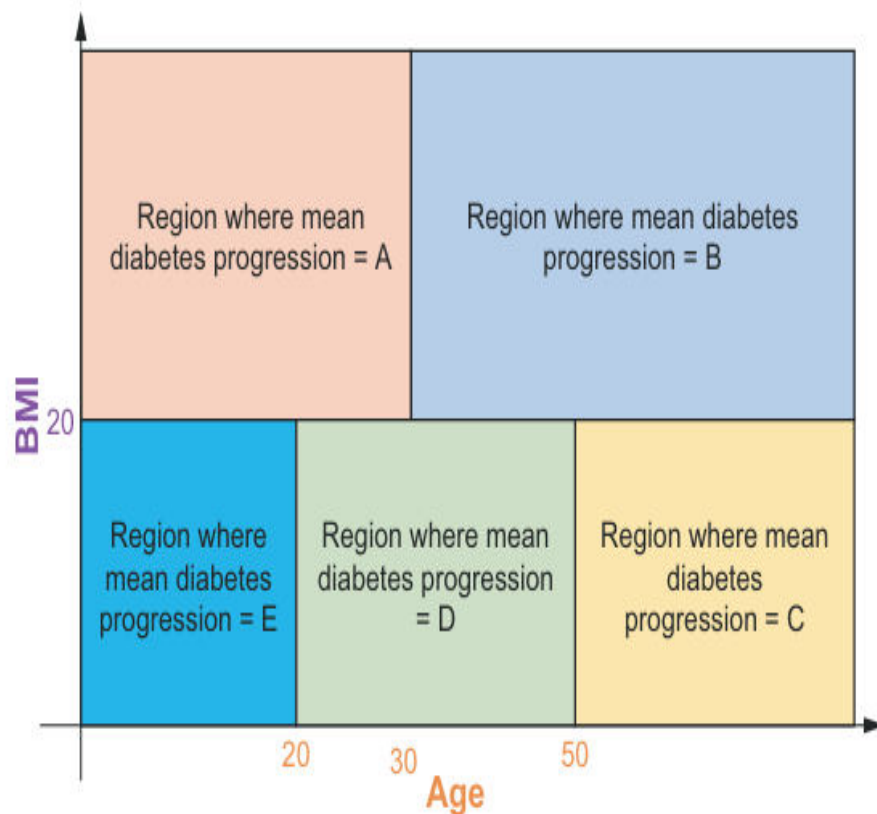
Explanation: Decision Trees split data into branches based on feature thresholds. Each branch leads to a prediction about disease risk.

Visualization: Tree diagram showing splits on age, BMI, and blood pressure.

Layman's View: Like a doctor's flowchart: "If age > 50 and BP $> 130 \rightarrow$ higher risk."

Decision Tree: Case Study

- Consider the previous diabetic example by considering only two features, BMI and Age. The decision tree splits the dataset into five groups in total, three age groups and two BMI groups



- The algorithm that is commonly applied in determining the optimum splits is the **classification and regression tree (CART)** algorithm.
- This algorithm first chooses a feature and a threshold for that feature. Based on that feature and threshold, the algorithm splits the dataset into the following two subsets:
 - **Subset 1, where the value of the feature is less than or equal to the threshold**
 - **Subset 2, where the value of the feature is greater than the threshold**

The algorithm picks the feature and threshold that minimizes a cost function or criterion. For **regression tasks**, this criterion is typically the **mean squared error (MSE)**, and for **classification tasks**, it is typically either **Gini impurity or entropy**.

The algorithm then continues to recursively split the data until the criterion is reduced further or until a maximum depth is reached

Case Study : Implementation as Binary Tree

- A decision tree model can be trained in Python using the Scikit-Learn package as follows. The code to learn the open diabetes dataset and to split it into the training and test sets is the same as the one used for linear regression

```
Trains the decision tree model
from sklearn.tree import DecisionTreeRegressor
dt_model = DecisionTreeRegressor(max_depth=None, random_state=42)
dt_model.fit(X_train, y_train)
y_pred = dt_model.predict(X_test)
mae = np.mean(np.abs(y_test - y_pred))
```

Imports the scikit-learn class for the decision tree regressor

Initializes the decision tree regressor. It is important to set the random_state to ensure that consistent, reproducible results can be obtained.

Uses the trained decision tree model to predict the disease progression for patients in the test set

Evaluates the model performance using the mean absolute error (MAE) metric

Decision Tree for Classification

```
from sklearn.tree import DecisionTreeClassifier
dt_model = DecisionTreeClassifier(criterion='gini',
max_depth=None)
dt_model.fit(X_train, y_train)
```

The criterion parameter in the DecisionTreeClassifier can be used to specify the cost function for the CART algorithm. By default, it is set to gini, but it can be changed to entropy

Visualization of the Diabetes Decision Tree Model:

```
from sklearn.externals.six import StringIO
from IPython.display import Image
from sklearn.tree import export_graphviz
import pydotplus
```

Imports all the necessary libraries to generate and visualize the binary tree

```
diabetes_dt_dot_data = StringIO()
export_graphviz(dt_model,
                out_file=diabetes_dt_dot_data,
                filled=False, rounded=True,
                feature_names=feature_names,
                proportion=True,
                precision=1,
                special_characters=True)
```

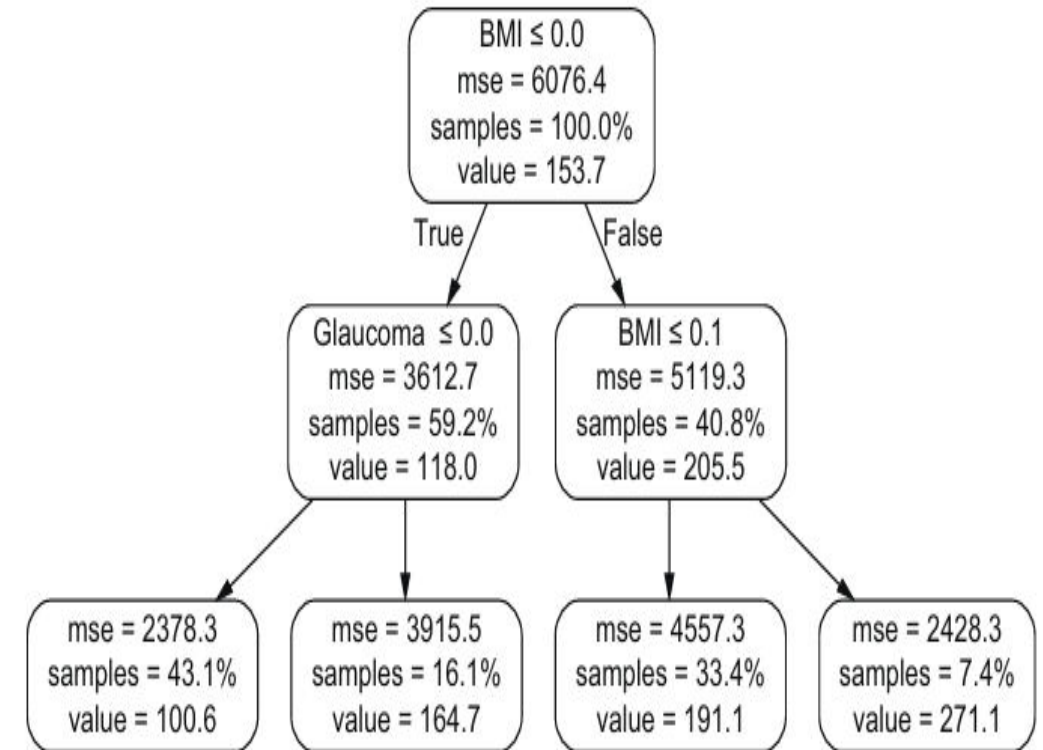
Initializes a string buffer to store the binary tree/graph in DOT format

Exports the decision tree model as a binary tree in DOT format

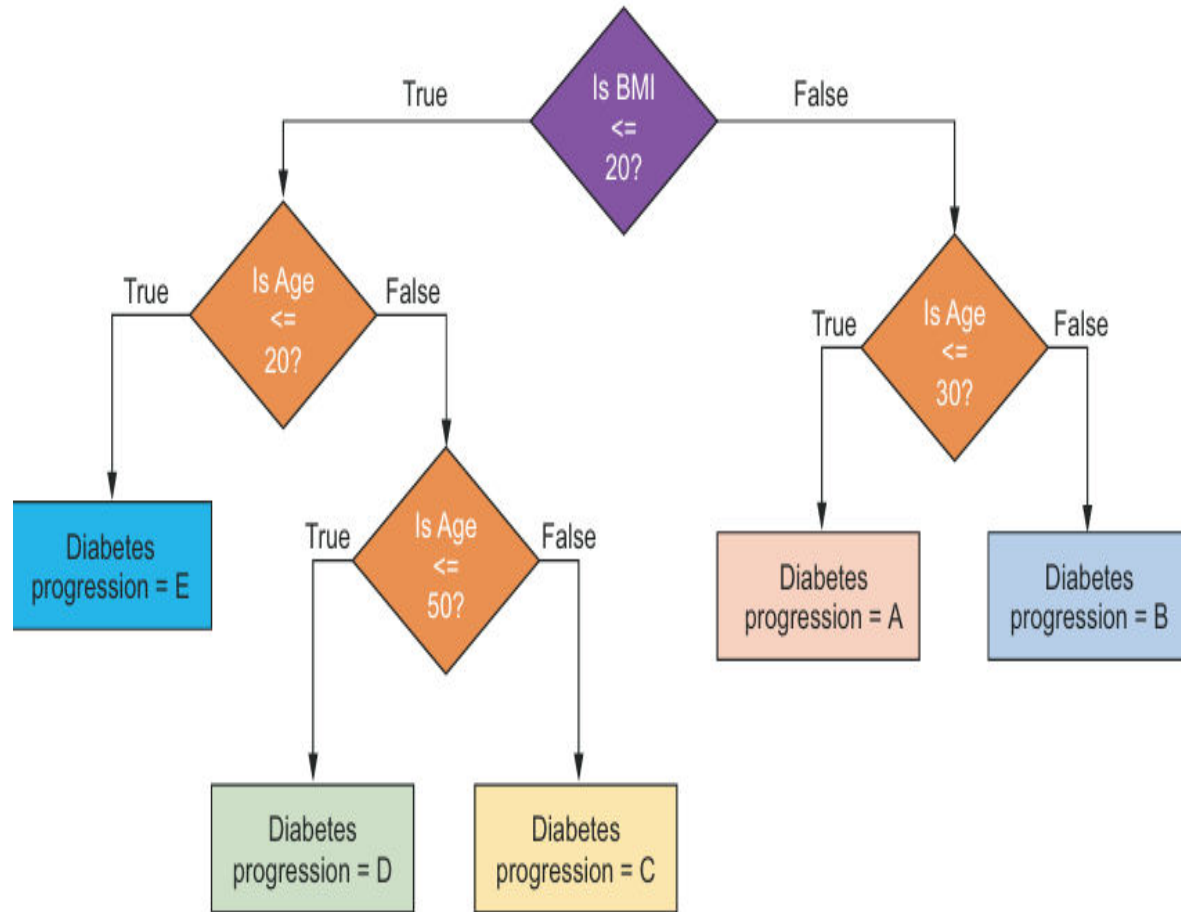
```
dt_graph = pydotplus.graph_from_dot_data(diabetes_dt_dot_data.getvalue())
Image(dt_graph.create_png())
```

Generates an image of the binary tree using the DOT format string

Visualizes the binary tree using the Image class



Interpreting Decision Trees :



A decision tree can be visualized as a bunch of if-else conditions strung together, where each condition splits the data in two. Such a model can be easily visualized as a binary tree. The tree can be interpreted as follows.

Starting at the root of the tree, check if the normalized BMI is ≤ 0 . If true, go to the left part of the tree. If false, go to the right part of the tree. Because we are starting at the root of the tree, this node accounts for 100% of the data. This is why samples is equal to 100%. Also, if we were to set the max_depth to 0 and predict the disease progression, then we would use the average value of all the samples in the data, which is 153.7, represented as value in the tree. By predicting 153.7, we would get an MSE of 6076.4.

If the normalized BMI is ≤ 0 , then we go to the left part of the tree and check if the normalized Glaucoma is ≤ 0 . If BMI is ≤ 0 , we would account for approximately 59% of the data, and the MSE would reduce from 6076.4 for the parent node to 3612.7. We can repeat this process until we have reached the leaf nodes in the tree. If we look at, say, the right-most leaf node, this corresponds to the following condition: if BMI > 0 and BMI > 0.1 and LDL > 0 , then predict 225.8 for 2.3% of the data, resulting in an MSE of 2757.9.

The complexity of this tree will increase as max_depth increases or as the number of input feature increases

Computing Feature importance:

To compute the feature importance, we first need to compute the importance of a node in the binary tree. The importance of a node is computed as the decrease in the cost function or impurity measure for that node weighted by the probability of reaching that node in the tree. Python code is given below:

$$I_k^{\text{node}} = \underbrace{p_k}_{\text{Proportion of samples to reach node } k} \cdot \underbrace{m_k}_{\text{Impurity measure of node } k} - \underbrace{p_k^{(\text{left})}}_{\text{Proportion of samples to reach left subtree of node } k} \cdot \underbrace{m_k^{(\text{left})}}_{\text{Impurity measure of left subtree of node } k} - \underbrace{p_k^{(\text{right})}}_{\text{Proportion of samples to reach right subtree of node } k} \cdot \underbrace{m_k^{(\text{right})}}_{\text{Impurity measure of right subtree of node } k}$$

→

$$I_i^{\text{feature}} = \frac{\sum_{j \in \mathbb{J}} I_j^{\text{node}}}{\sum_{k \in \mathbb{K}} I_k^{\text{node}}}$$

Importance of feature i = $\frac{\text{Sum of importance of all nodes } j \text{ that split on feature } i}{\text{Sum of importance of all nodes } k \text{ in the decision tree}}$

Gets feature importance from the trained decision tree model

```
weights = dt_model.feature_importances_
```

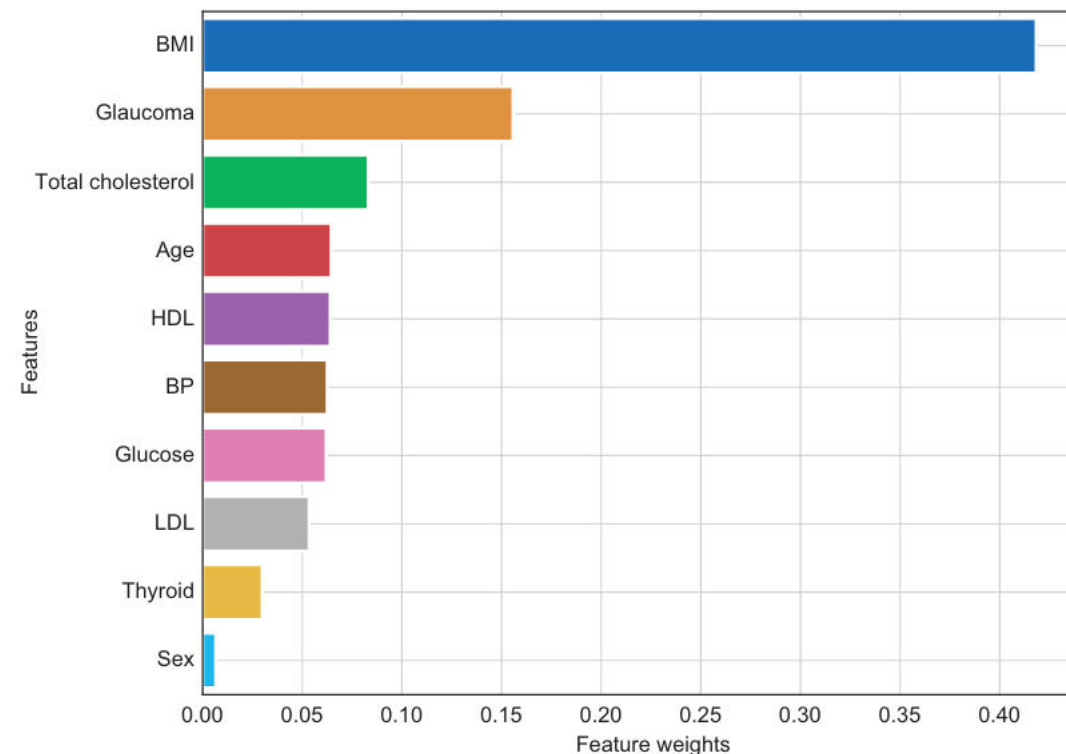
Sorts indices of feature weights in descending order of importance

Gets the feature names and feature weights in descending order of importance

```
feature_importance_idx = np.argsort(np.abs(weights))[:, ::-1]
feature_importance = [feature_names[idx].upper() for idx in feature_importance_idx]
feature_importance_values = [weights[idx] for idx in feature_importance_idx]
```

```
f, ax = plt.subplots(figsize=(10, 8))
sns.barplot(x=feature_importance_values, y=feature_importance, ax=ax)
ax.grid(True)
ax.set_xlabel('Feature Weights')
ax.set_ylabel('Features')
```

Generates the plot shown in figure 2.14



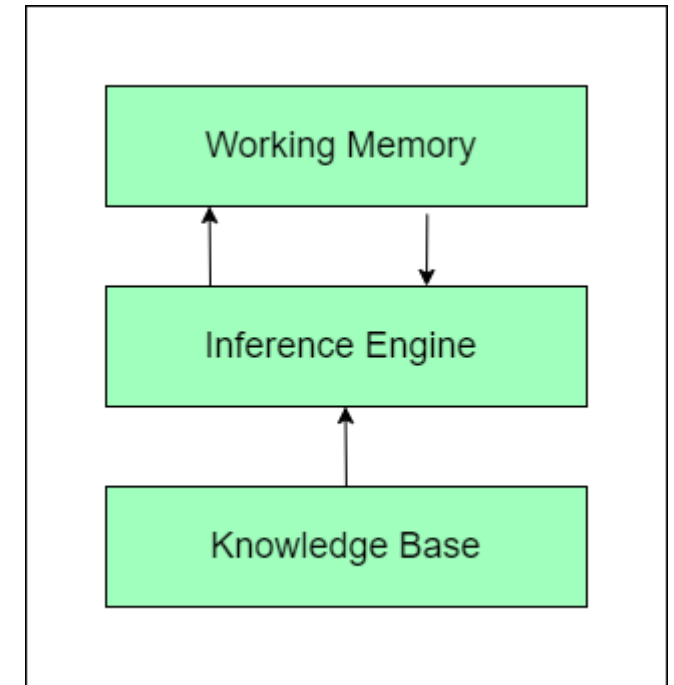
RULE BASED METHODS:

- Rule-based methods are a core Explainable AI (XAI) approach that use explicit "if-then" logic to make transparent, human-understandable decisions, forming intrinsically interpretable models like Decision Trees or acting as simpler "**surrogate**" models to explain complex "black-box" systems

- **They provide clarity by showing exactly which conditions triggered a conclusion, making them ideal for critical domains like healthcare and finance where tracing decisions is vital**

There are many ways to learn rules from data:

1. **OneR** : learns rules from a single feature. OneR is characterized by its simplicity, interpretability and its use as a benchmark. A OneR model is a decision tree with only one split. The split is not necessarily binary as in CART, but depends on the number of unique feature values
2. **Sequential covering** : is a general procedure that iteratively learns rules and removes the data points that are covered by the new rule. This procedure is used by many rule learning algorithms.
3. **Bayesian Rule Lists**: combine pre-mined frequent patterns into a decision list using Bayesian statistics. Using pre-mined patterns is a common approach used by many rule learning algorithms



ANCHORS(Scoped Rules) :

- In Explainable AI (XAI), Anchors are high-precision, model-agnostic IF-THEN rules that identify sufficient conditions (the "anchor") for a model's prediction, ensuring that if the anchor conditions hold, the prediction stays the same, regardless of other feature changes, making explanations clear and reusable for various data types like text, image, and tabular data
- Anchors utilizes reinforcement learning techniques in combination with a graph search algorithm to reduce the number of model calls (and hence the required runtime) to a minimum while still being able to recover from local optima
- This method is therefore considered “**Local**”, “**Post hoc**” and “**Model agnostic**”

How Anchors Work:

- **Perturbation-Based:** Like LIME, Anchors perturb the input data to find rules.
- **Rule Generation:** They use reinforcement learning and graph search to find minimal sets of feature conditions (predicates) that, when met, lead to a specific high-precision outcome.
- **High Precision & Coverage:** An anchor rule is a set of conditions (e.g., "IF age > 40 AND income < 50k") that strongly predicts an outcome, with "coverage" indicating how many instances this rule applies to. 

Anchor : Example

For instance, suppose we are given a bivariate black box model that predicts whether or not a passenger survived the Titanic disaster. Now we would like to know why the model predicts for one specific person (see Table) that they survived. The anchors algorithm provides a result explanation like the one shown below.

And the corresponding anchors explanation is:

IF SEX = female AND Class = first THEN PREDICT Survived = true WITH PRECISION 97% AND COVERAGE 15%

An anchor A is formally defined as follows:

$$\mathbb{E}_{\mathcal{D}_{\mathbf{x}}(\mathbf{z}|A)}[1_{\hat{f}(\mathbf{x})=\hat{f}(\mathbf{z})}] \geq \tau; A(\mathbf{x}) = 1$$

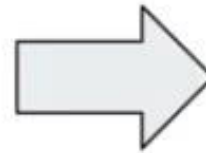
Given an instance $\mathbf{x}$ to be explained, a rule or an anchor A is to be found, such that it applies to $\mathbf{x}$, while the same class as for $\mathbf{x}$ gets predicted for a fraction of at least τ of $\mathbf{x}$'s neighbors where the same A is applicable. A rule's precision results from evaluating neighbors or perturbations (following $\mathcal{D}_{\mathbf{x}}(\mathbf{z}|A)$) using the provided machine learning model (denoted by the indicator function $1_{\hat{f}(\mathbf{x})=\hat{f}(\mathbf{z})}$).

Feature	Value
Age	20
Sex	female
Class	first
Ticket price	300\$
More attributes	...
Survived	true

Anchor example



Feature	Value
Age	20
Sex	female
Class	first
Ticket Price	300\$
More attributes	...
Survived	true



```
IF SEX = female  
AND Class = first  
THEN PREDICT Survived = true  
WITH PRECISION 97%  
AND COVERAGE 15%
```

Anchors : Formal Definition

An anchor A is formally defined as follows:

$$\mathbb{E}_{\mathcal{D}_{\mathbf{x}}(\mathbf{z}|A)}[1_{\hat{f}(\mathbf{x})=\hat{f}(\mathbf{z})}] \geq \tau; A(\mathbf{x}) = 1$$

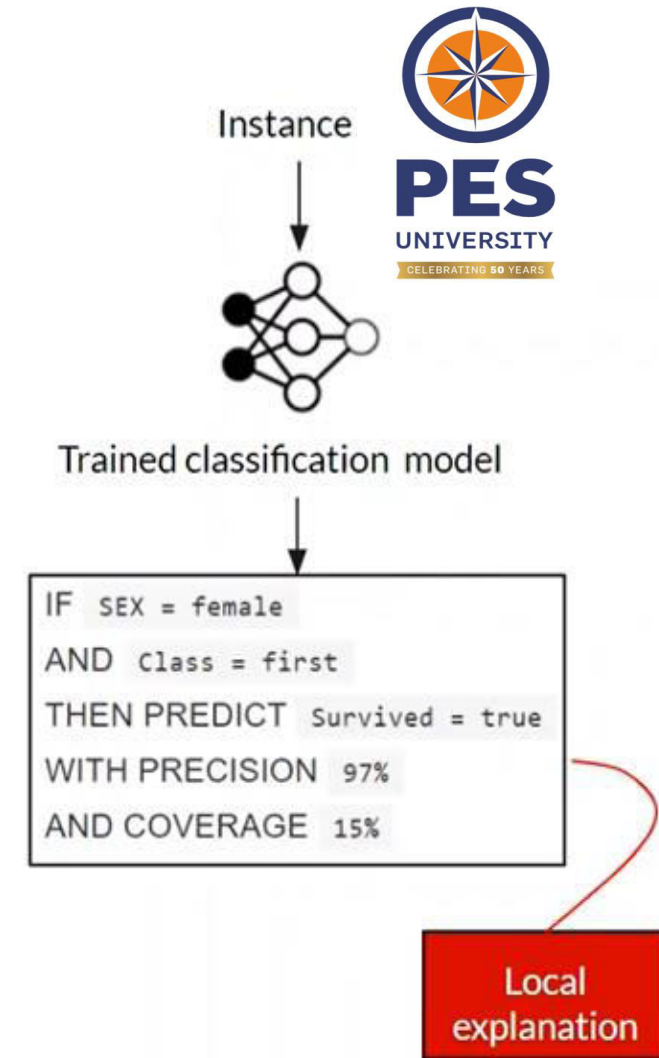
Wherein:

- $\mathbf{x}$ represents the instance being explained (e.g. one row in a tabular dataset).
- A is a set of predicates, i.e., the resulting rule or anchor, such that $A(\mathbf{x}) = 1$ when all feature predicates defined by A correspond to $\mathbf{x}$'s feature values.
- $\hat{f}$ denotes the classification model to be explained (e.g. an artificial neural network model). It can be queried to predict a label for $\mathbf{x}$ and its perturbations.
- $\mathcal{D}_{\mathbf{x}}(\cdot|A)$ indicates the distribution of neighbors of $\mathbf{x}$, matching A .
- $0 \leq \tau \leq 1$ specifies a precision threshold. Only rules that achieve a local fidelity of at least τ are considered a valid result.

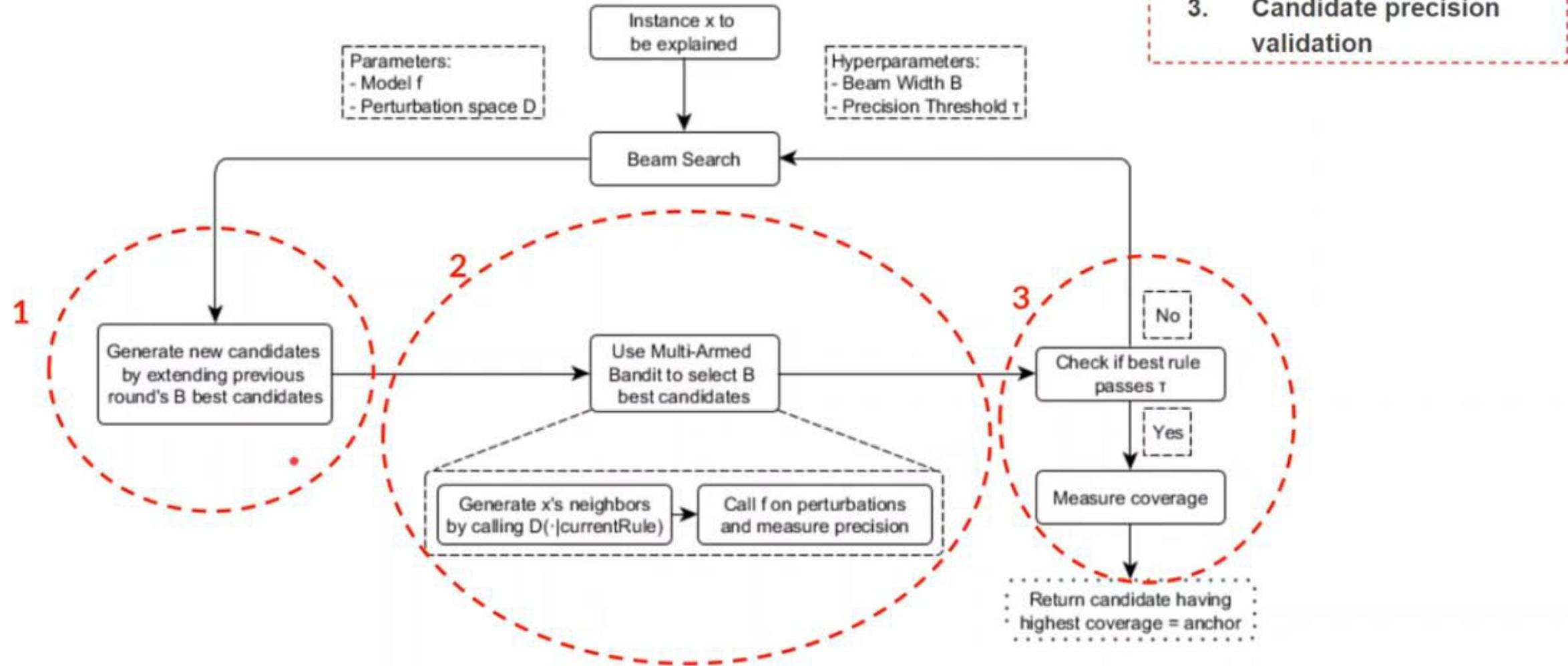
Given an instance $\mathbf{x}$ to be explained, a rule or an anchor A is to be found, such that it applies to $\mathbf{x}$, while the same class as for $\mathbf{x}$ gets predicted for a fraction of at least τ of $\mathbf{x}$'s neighbors where the same A is applicable. A rule's precision results from evaluating neighbors or perturbations (following $\mathcal{D}_{\mathbf{x}}(\mathbf{z}|A)$) using the provided machine learning model (denoted by the indicator function $1_{\hat{f}(\mathbf{x})=\hat{f}(\mathbf{z})}$).

Anchors (Scoped Rules)

- What is?
 - Anchors is a model-agnostic method that explains individual predictions
- How does it work?
 - It uses a perturbation-based strategy to generate local explanations
- What is the input?
 - Instance x to be explained, trained black-box model
- What is the output?
 - Easy to understand IF-THEN rules (called Anchors) with precision and coverage
- What techniques does it use?
 - Data perturbation (same as LIME)
 - Reinforcement learning (Multi-armed bandit)
 - Graph search algorithm (Beam search)



Anchors algorithm's components



Anchor – Example Implementation

To test the Anchor method (in Python), we focus on the “Red Wine Quality” dataset with physicochemical inputs and **quality** as output

Fixed acidity	Volatile acidity	Citric acid	Residual sugar	Chlorides	Free sulfur dioxide	Total sulfur dioxide	Density	pH	Sulphates	Alcohol	Quality
7,4	0,7	0	1,9	0,076	11	34	0,9978	3,51	0,56	9,4	5
7,8	0,88	0	2,6	0,098	25	67	0,9968	3,2	0,68	9,8	5
7,6	0,71	0,04	2,3	0,082	15	54	0,997	3,26	0,61	9,8	5
11,2	0,28	0,56	1,9	0,075	17	60	0,998	3,16	0,58	9,8	6
7,4	0,7	0	1,9	0,076	11	34	0,9978	3,51	0,56	9,4	5
7,4	0,66	0	1,8	0,075	13	40	0,9978	3,51	0,56	9,4	5
7,9	0,6	0,06	1,8	0,089	15	60	0,9994	3,3	0,49	9,4	5
7,3	0,65	0	1,2	0,065	15	21	0,9946	3,39	0,47	10	7
7,3	0,58	0,02	2	0,073	9	16	0,9993	3,36	0,67	9,5	7
7,5	0,5	0,36	6,1	0,071	17	102	0,9978	3,35	0,8	10,5	5
6,7	0,53	0,06	1,9	0,097	15	65	0,9955	3,28	0,54	9,2	5

Then, we train a Random Forest model to predict the quality output.

```
import pandas as pd
from sklearn.metrics import accuracy_score
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier

df = pd.read_csv("./winequality-red.csv", sep=';')
feature_columns = df.columns[:-1]
X = df[feature_columns]
y = df['quality']
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size=0.2)
model = RandomForestClassifier(n_estimators=500)
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print('Accuracy of random forest: ', accuracy_score(y_test,y_pred))
```

We obtain the following result:

Accuracy of random forest: 0.709375

Anchor – Example Implementation

With that dataset and that trained model, we can apply the Anchor method and we get the following rule for a random point:

```
IF 'alcohol' > 11.10 AND 'total sulfur dioxide' > 22.00  
THEN PREDICT 'quality' = 6  
WITH Precision = 0.7201 AND Coverage = 0.1432
```

In addition to the generated prediction rule, two metrics are also returned that provide insight about the quality and the relevance of the rule:

- Precision:** the relative number of correct predictions. In the previous result, the rule was correct 7.2 times on average for 10 predictions.
- Coverage:** the “perturbation space” (or the anchor’s probability of applying to its “neighbors”) or the relative number of points generated by the “perturbation” that follow the rule. Generally, the more variables the rule contains, the lower the coverage

Exploration of models with Anchor method:

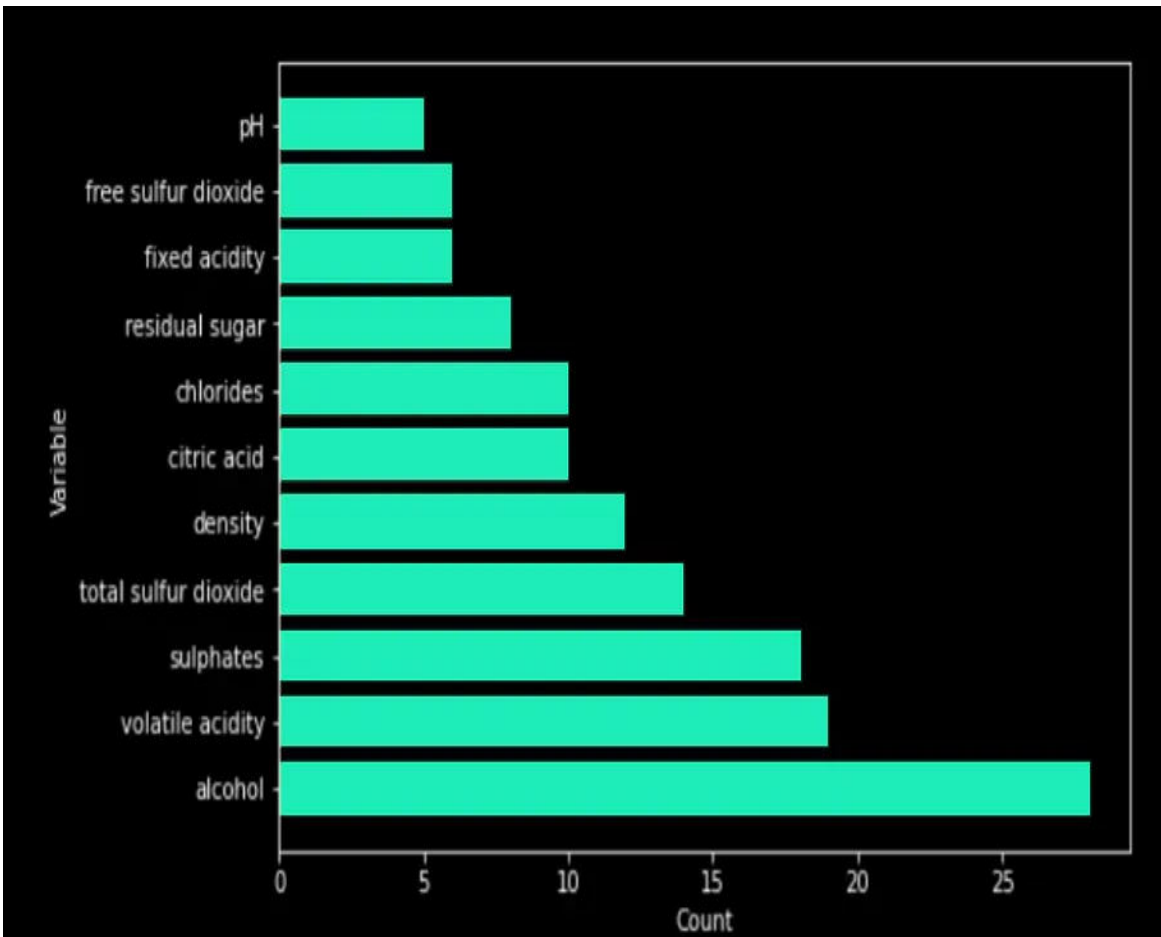
- From that interpretability method, it is quite possible to explore and understand any prediction model.
- To do so, we will iterate the Anchor method over many points of the dataset.
- The algorithm takes as input a minimal precision threshold for Anchor method and a proportion of points of the dataset implied by all the rules and as output, the list of all the rules.
- About the exploration, the principle is the following:
 1. Get a random point in the dataset
 2. Apply Anchor method on that point
 3. Remove the points affected by the rule from the dataset
 4. Save the rule
 5. Repeat from step 1 until less than 20% of points remain

By applying the exploration algorithm described above with a minimum accuracy threshold of 0.7 and by traversing at least 80% of the dataset, we generate more than thirty prediction rules. Here are some of them

```
IF 'sulphates' > 0.73 AND 'volatile acidity' <= 0.39 AND 'alcohol' > 10.20 AND 'citric acid' <= 0.26
AND 'density' <= 1.00 AND 'pH' <= 3.31 AND 'fixed acidity' > 9.20 AND 'free sulfur dioxide' <= 21.00
AND 'chlorides' <= 0.08
THEN PREDICT 'quality' = 7
WITH Precision = 0.719 AND Coverage = 0.005
IF 'alcohol' > 10.20 AND 'residual sugar' <= 2.20 AND 'density' > 1.00 AND 'volatile acidity' <= 0.39
THEN PREDICT 'quality' = 6
WITH Precision = 0.762 AND Coverage = 0.006
IF 'alcohol' > 11.10 AND 'sulphates' <= 0.73 AND 'volatile acidity' <= 0.52
THEN PREDICT 'quality' = 6
WITH Precision = 0.739 AND Coverage = 0.1014
IF 'alcohol' <= 10.20 AND 'total sulfur dioxide' > 62.00
THEN PREDICT 'quality' = 5
WITH Precision = 0.757 AND Coverage = 0.181
IF 'sulphates' <= 0.55 AND 'alcohol' <= 11.10 AND 'volatile acidity' > 0.39
THEN PREDICT 'quality' = 5
WITH Precision = 0.705 AND Coverage = 0.192
```


Exploration of models with Anchor method:

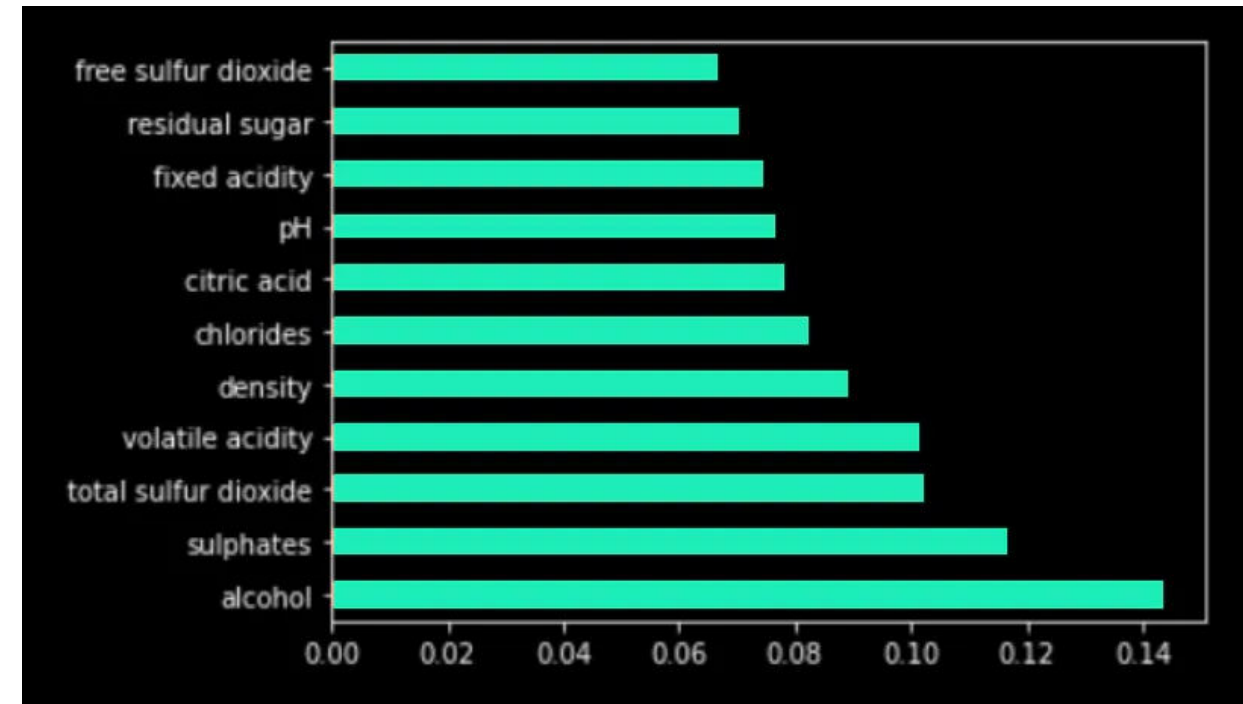
To check the relevance of those rules, the variables involved in the rules are compared directly with the results of a feature importance. First, we count the **number of occurrences** of each variable in the rules.



Then, secondly, we run a Feature Importance algorithm on the Random Forest.



```
feature_importances = pd.Series(model.feature_importances_, index=X.columns)
feature_importances.nlargest(15).plot(kind='barh')
```



Rule-Based Anchors (House Prices Dataset)

Concept: Anchors (rule-based explanations)

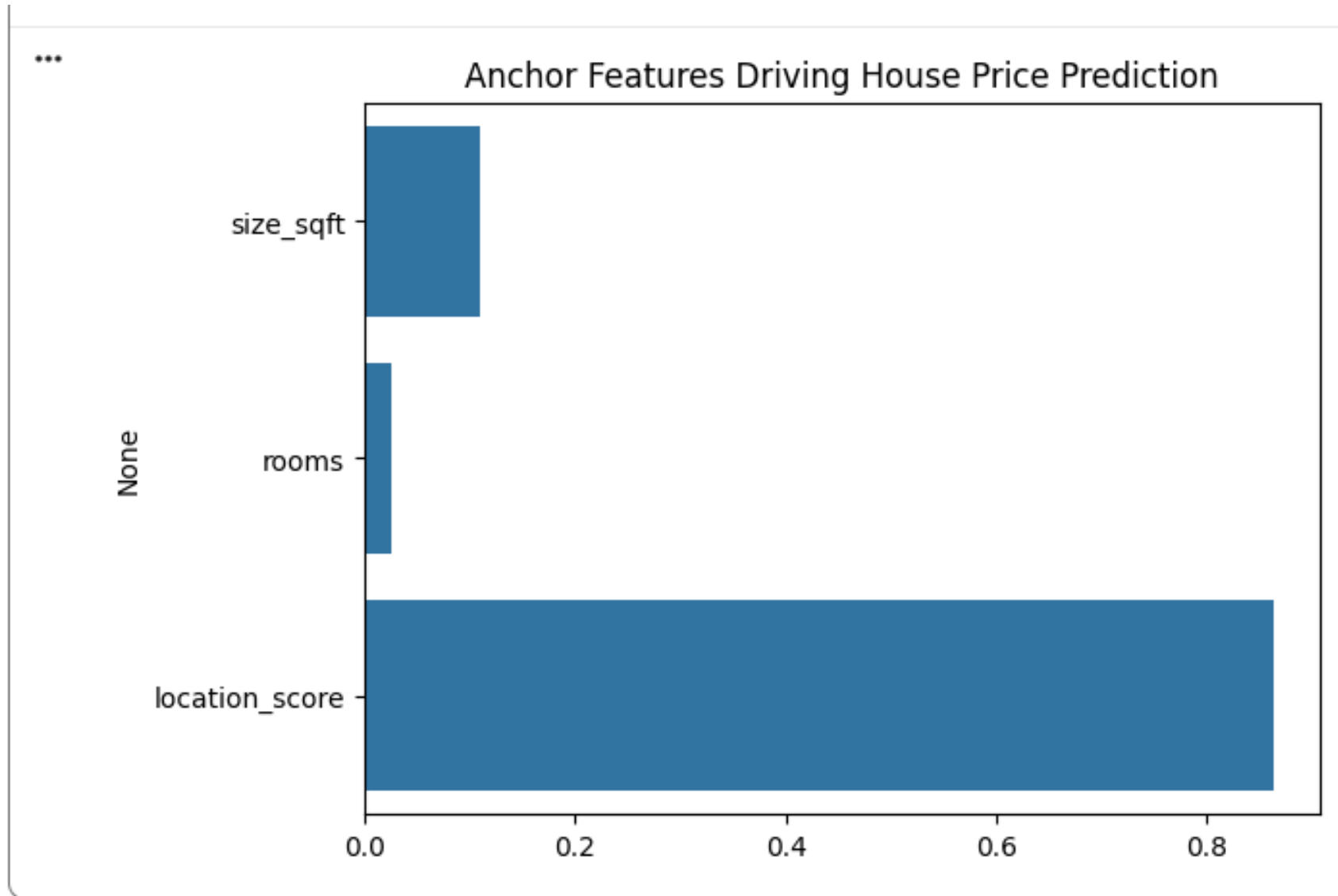
Dataset: house_prices.csv (size, rooms, location_score, price)

Explanation: Anchors are simple rules that strongly influence predictions. Example: “If $\text{location_score} > 0.8$, price is high regardless of other features.”

Visualization: Bar chart of feature importance showing anchors like location_score and size_sqft.

Layman’s View: Anchors are golden rules: “Big house in good location → expensive.”

ANCHORS - Practical



COUNTERFACTUAL EXPLANATIONS (Telecom Churn Dataset)



Concept: Counterfactuals (“what if” scenarios)

Dataset: telecom_churn.csv (tenure, charges, contract_type, churn)

Explanation: Counterfactuals show how changing one feature alters the prediction.
Example: “If contract_type changes from monthly to yearly, churn prediction flips.”

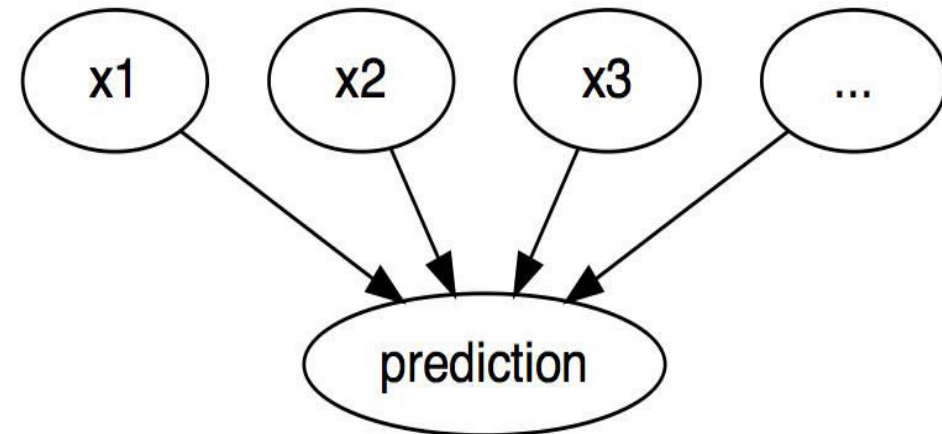
Visualization: Bar chart comparing original vs counterfactual prediction.

Layman’s View: Counterfactuals answer: “What would need to change for a different outcome?”

COUNTER FACTUAL EXPLANATIONS :

- **A counterfactual explanation of a prediction describes the smallest change to the feature values that changes the prediction to a predefined output.**
 - A counterfactual explanation of an outcome or a situation takes the form of “**If X had not occurred, Y would not have occurred**”.
 - Example: “**If I hadn’t taken a sip of this hot coffee, I wouldn’t have burned my tongue.**” Event Y is that I burned my tongue; cause X is that I had a hot coffee.
 - Thinking in counterfactuals requires imagining a hypothetical reality that contradicts the observed facts (for example, a world in which I’ve not drunk the hot coffee), hence the name “counterfactual.”
- The ability to think in counterfactuals makes us humans so smart
- compared to other animals..

- In interpretable machine learning, counterfactual explanations can be used to explain predictions of individual instances.
- **The relationship between the inputs and the prediction is very simple: The feature values cause the prediction.**



How It works:

Minimal Perturbation: They find the smallest, most realistic adjustments to an instance's features (e.g., income, age, BMI) that result in a different prediction (e.g., loan approval, non-diabetic).

Contrastive Nature: They contrast the original, rejected outcome with a hypothetical, successful alternative.

Example: If a loan is denied due to low income, a counterfactual might be: "If your income was \$5,000 higher, your loan would have been approved"

Key Benefits:

Actionable Insights: Users know what to change to achieve a different result, providing concrete steps for recourse.

Trust & Debugging: Help users trust decisions and allow providers to check for fairness, bias, or model robustness, acting as a debugging tool.

Human-Friendly: They align with how humans reason causally about hypothetical situations, making them easy to understand

✓ **Challenge! –**

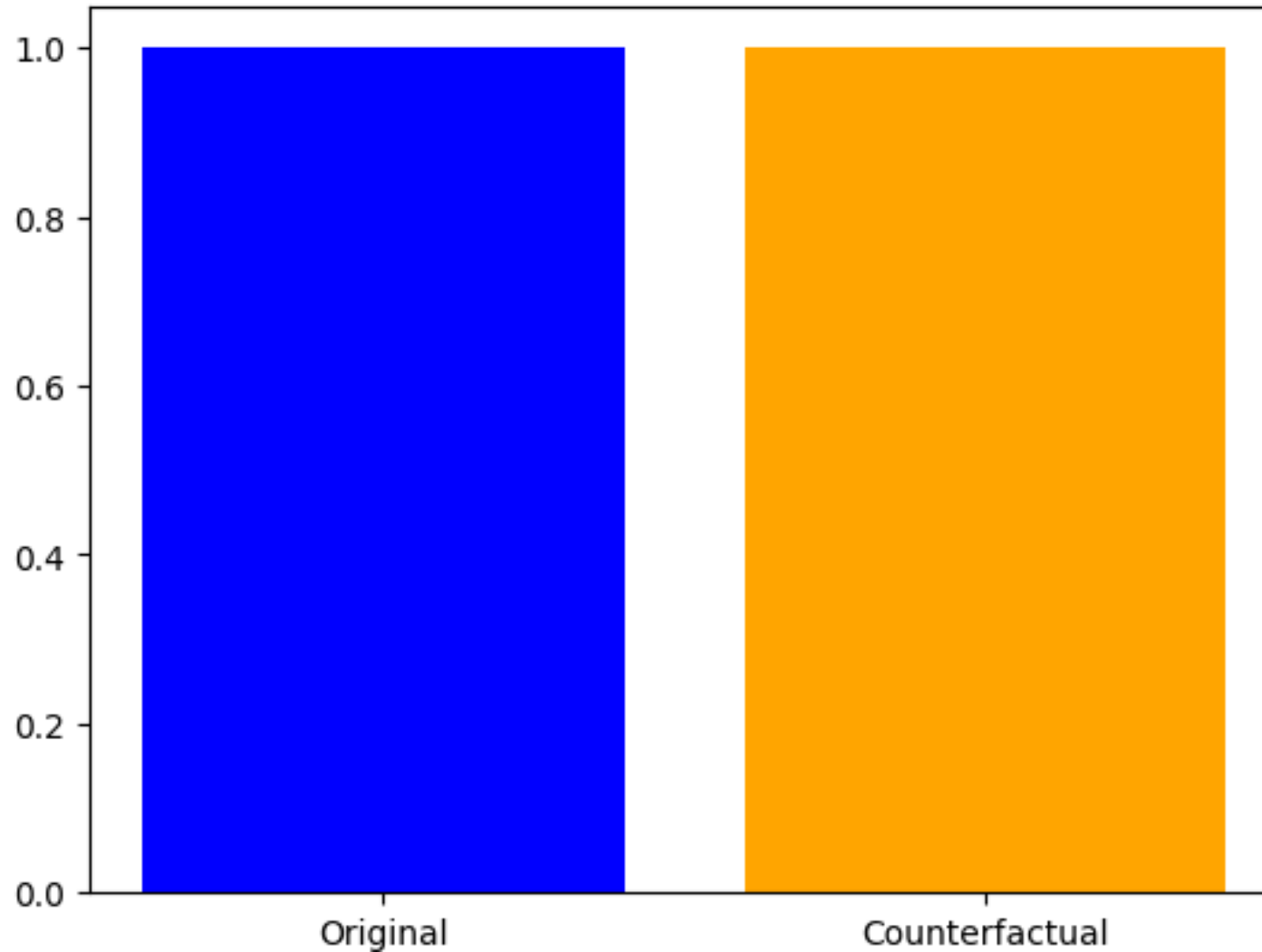
Rashomon Effect: Multiple valid, contradictory counterfactuals can exist, requiring methods to select the "best" or most relevant one

Causality vs. Correlation: While intuitive, AI models often learn correlations, not true causation

Counter Factual Explanations – Practical

Original prediction: 1
*** Counterfactual prediction (changed contract_type): 1

Counterfactual Explanation: Effect of Contract Type on Churn



Counter Factual Explanations – Usecase

- Peter applies for a loan and gets rejected by the (machine learning-powered) banking software. He wonders why his application was rejected and how he might improve his chances to get a loan.
- The question of “why” can be formulated as a counterfactual: What’s the smallest change to the features (income, number of credit cards, age, ...) that would change the prediction from rejected to approved?
- One possible answer could be: **If Peter would earn 10,000 more per year, he would get the loan.** Or **if Peter had fewer credit cards and had not defaulted on a loan five years ago, he would get the loan.** Peter will never know the reasons for the rejection, as the bank has no interest in transparency, but that is another story.
- [15 Counterfactual Explanations – Interpretable Machine Learning](#)

STRUCTURED DATA EXPLAINERS

Concept: Structured Data Explainers (feature importance)

Dataset: insurance_data.csv (driver_age, mileage, prior_accidents, claim)

Explanation: Structured explainers show which features most influence predictions. Example: prior_accidents and mileage are strong predictors of claims.

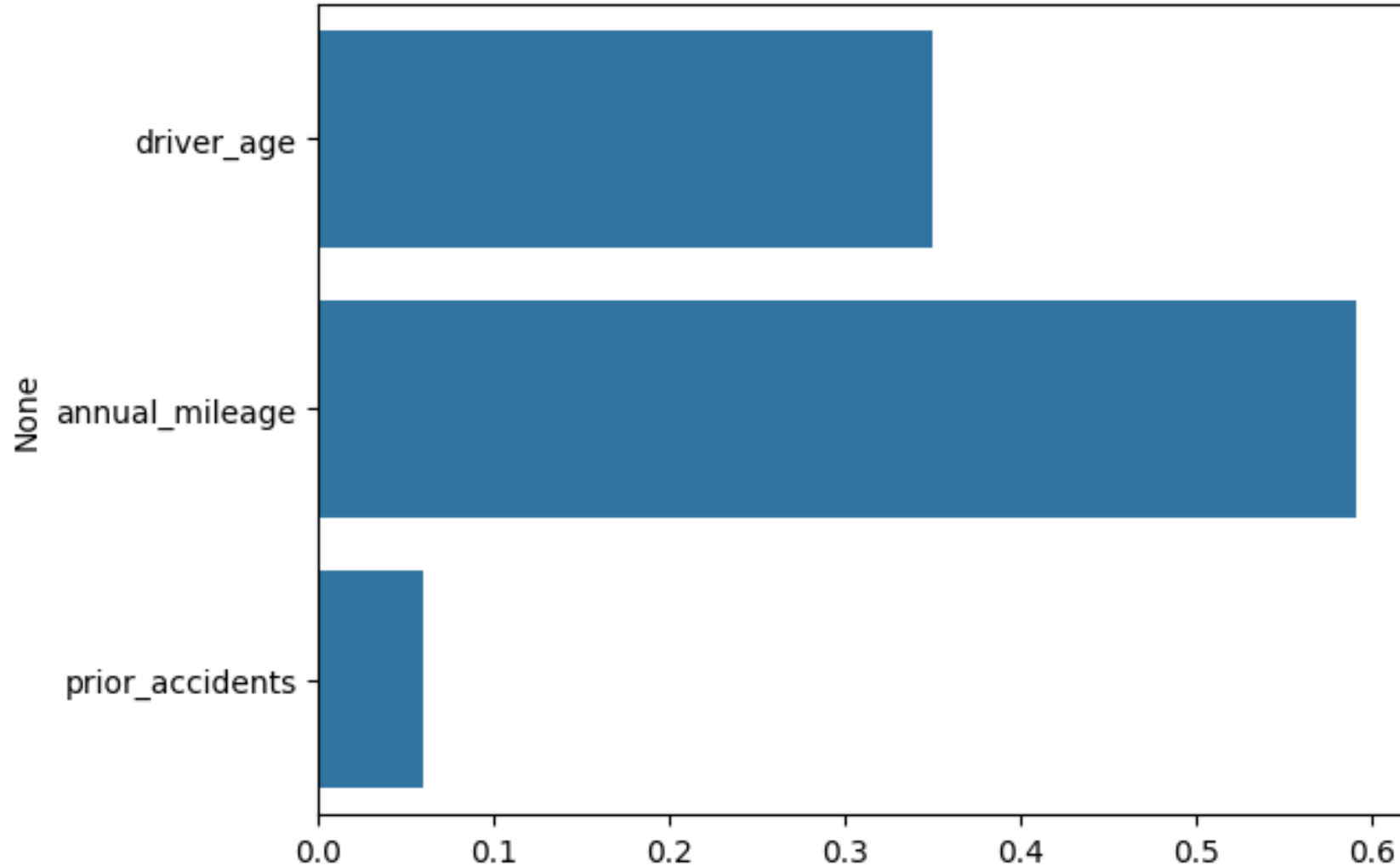
Visualization: Bar chart of feature importance.

Layman's View: Like ranking risk factors: “More accidents and higher mileage → higher claim probability”

Structured Data Explainers - Practical

...

Structured Data Explainer: Feature Importance for Insurance Claims



Unit 1 – Recap part 1

1. Linear Regression

$$y = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \cdots + \beta_n x_n + \epsilon$$

2. Logistic Regression

$$P(y = 1|X) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_n x_n)}}$$

3. Generalized Linear Models (GLM)

$$g(\mu) = X\beta$$

4. Generalized Additive Models (GAM)

$$y = \beta_0 + f_1(x_1) + f_2(x_2) + \cdots + f_n(x_n) + \epsilon$$

Unit 1 – Recap part 2

5. LIME (Local Interpretable Model-Agnostic Explanations)

$$\operatorname{argmin}_g \sum_{z \in Z} \pi_x(z) \cdot (f(z) - g(z))^2$$

6. Counterfactual Explanations

$$x' = \operatorname{argmin}_{x'} d(x, x') \quad \text{s.t.} \quad f(x') \neq f(x)$$

7. Decision Tree Splits

IF $x_i \leq t$ THEN go left, ELSE go right

Linear Regression

Equation: $y = \beta_0 + \beta_1x_1 + \beta_2x_2 + \dots + \beta_nx_n + \varepsilon$

Coefficients: Each β_i shows how much the target changes when feature x_i increases by 1 unit.

Intercept (β_0): Baseline prediction when all features are zero.

XAI Justification: Intrinsically interpretable — coefficients directly explain predictions.

Example: Each extra square foot adds ₹245 to house price.

Logistic Regression

Equation: $P(y=1|X) = 1 / (1 + e^{\{-(\beta_0 + \beta_1 x_1 + \dots + \beta_n x_n)\}})$

Coefficients: Represent change in log-odds of outcome per unit increase in x_i .

Sigmoid Function: Maps linear combination into probability between 0 and 1.

XAI Justification: Coefficients can be interpreted as odds ratios.

Example: Each year of age increases disease odds by 5%.

Generalized Linear Models (GLM)

Equation: $g(\mu) = X\beta$

Link Function (g): Connects predictors to expected value μ .

Coefficients: Meaning depends on chosen link (logit, log, etc.).

XAI Justification: Extends interpretability to diverse data types while keeping transparency.

Example: Poisson regression with log link: each extra accident increases expected claims multiplicatively.

Generalized Additive Models (GAM)

Equation: $y = \beta_0 + f_1(x_1) + f_2(x_2) + \dots + f_n(x_n) + \varepsilon$

Functions (f_i): Smooth curves showing how each feature contributes.

Intercept (β_0): Baseline prediction.

XAI Justification: Allows non-linear relationships but remains interpretable by visualizing each feature's effect.

Example: BMI curve: risk rises steeply until BMI=30, then levels off.

Counterfactual Explanations

Equation: $x' = \operatorname{argmin}_{\{x'\}} d(x, x')$ subject to $f(x') \neq f(x)$

Original instance (x): Current input.

Counterfactual (x'): Closest alternative input that changes prediction.

Distance function (d): Ensures minimal change.

XAI Justification: Provides actionable insights — “what needs to change for a different outcome.”

Example: Loan denied at GPA=2.8 → Counterfactual says GPA=3.2 flips to approval.

Decision Tree Splits

Equation/Rule: IF $x_i \leq t$ THEN go left ELSE go right

Threshold (t): Value at which feature splits.

Path Rules: Sequence of splits leads to prediction.

XAI Justification: Fully transparent — every prediction can be traced through rules.

Example: If age > 50 and BP > 130 → predict disease.



THANK YOU
